BUILD WITH AI: DEVOPS 2026 · GDG ON CAMPUS, DEEN DAYAL UPADHYAYA GORAKHPUR UNIVERSITY

Smart Campus Classroom Management

A corrected design for room allocation across the Institute of Engineering and Technology and Deeksha Bhawan

This report records a design decision, not only a design. It restates the problem from the two buildings upward, examines the proposal the team first sketched, identifies the four places where that proposal would have failed, and then rebuilds the model around intervals, database-enforced conflict prevention and a clean separation between allocation and the academic record.

30	32	2	0
bookable rooms across both buildings	sections, every one 75 strong	rooms that seat a spaced examination	double bookings the database will allow

DOCUMENT	Corrected design specification, issue 1.0, prepared for the Build With AI: DevOps 2026 evaluation panel
PREPARED BY	The project team: data and engine, application programming interface, interface design, delivery
SOURCES	The supplied problem statement and the architect's floor plan of both buildings supplied with it
STATUS	Complete for implementation. The five points where the sources disagree are set out in section 6.5 and answered in section 14

Fifteen sections · eighteen figures · twenty-nine tables · every figure derived from the supplied register

§

NAVIGATION

Contents

1 Purpose, scope and how to read this report

2 The problem as the campus experiences it

3 The proposal we began with, and the four defects in it

4 The corrected model: rooms are timelines, not switches

5 The three roles and how each of them works

6 Database design

7 Data flow

8 Comparison with the methods in use today

9 Comparative figures

10 Edge cases

11 Quality: non-functional requirements and testing

12 Delivery: repository, pipeline and deployment

13 Risks and mitigations

14 Frequently asked questions

15 Appendices: room register, diagram sources, derivations, glossary

HOW TO READ THIS REPORT

Part one establishes the problem and puts the team's first proposal under examination. Part two rebuilds the system and is the contract for whoever implements it. Part three carries the evidence: comparisons, arithmetic and the awkward cases. Part four covers delivery, risks and the questions we expect. Every claim is either sourced, calculated, or labelled as an assumption.

SUMMARY

At a glance

The short version, for anyone who reads only this page.

30 of 34
rooms are bookable; the rest are offices and one service space

93%
of all room-slots are needed in an examination week

2
rooms can seat a spaced 75-student examination

84%
chance of a clash per day when booking is uncoordinated

3
database constraints replace every application-level check

0
clashes the storage engine will permit once live

What we kept from the first proposal

The room register with a capacity for each room. Filtering by headcount as the primary workflow. The instinct that rooms return to a pool rather than being consumed. All three survive into the final design.

What we changed

A free-or-occupied flag became a timeline of reservations. Marking a room free became a status change with a reason. Checking for conflicts in application code became an exclusion constraint in the database. Capacity alone became nominal capacity plus examination capacity, purpose, features and walking distance.

What we removed

Attendance as an allocation signal. A cancelled class and an empty class are different facts, and conflating them is how the original incident became impossible to correct. Attendance remains a departmental record with no route into the booking table.

KEY The design turns a coordination problem into a constraint. Once the rule lives in the storage engine, no amount of simultaneous clicking, rushed scheduling or good intentions can produce two classes in one room.

PART I

The problem and the proposal

What the campus needs, what we first proposed, and why it would not have held.

— 1 Purpose, scope and how to read this report
— 2 The problem as the campus experiences it
— 3 The proposal we began with, and the four defects in it

SECTION 1

Purpose, scope and how to read this report

Section 2 does the groundwork: it states what the campus actually needs, using the dimensions of the two buildings as evidence rather than as background. Section 3 is the part most design documents leave out. It puts the team's first proposal on the page, gives it a fair hearing, and then explains, with a concrete counter-example, where it would have broken. Sections 4 to 7 rebuild the system from that point. Sections 8 and 9 answer the question an evaluation panel is entitled to ask: how much better is this than what happens today, and by how much. Section 10 collects the awkward cases. Everything after that is delivery material and reference.

We have tried to keep every claim checkable. Where a figure comes from the supplied blueprint or from the problem statement, it is marked as such. Where it comes from a calculation over those figures, the calculation is shown. Where it is an assumption we could not verify, it is labelled an assumption and given a range rather than a single number dressed up as a fact.

1.1 Who this is for

- **The implementation team.** Sections 4, 6 and 7 form the contract. If two members disagree about behaviour, the tie is broken here rather than in a chat thread.
- **The evaluation panel.** Sections 2, 3, 8 and 9 carry the argument; section 14 carries the questions we expect and the answers we would give.
- **A future maintainer.** The registry in Appendix A and the constraints in section 4.3 are enough to bring the system back to a known state without reading the source.

1.2 Terms used in this document

Term	Meaning in this document
Interval	A pair of instants, a start and an end, written [start, end). The end instant belongs to the next interval, so a reservation ending at 11:00 and one beginning at 11:00 do not collide.
Reservation	One row recording that a room is committed to one activity for one interval. In the database the table is called BOOKING.
Derived availability	The answer to "is this room free?", computed from reservations, the timetable and closures at the moment of asking. No row stores it.
Exclusion constraint	A PostgreSQL rule that rejects any new row whose room and interval duplicate an existing confirmed row's room and interval.
Section	A teaching group, for example CSE-3A. There are thirty-two of them across four years.
Term	A teaching spell of fifty-five minutes. Eight fit in a working day.
Blueprint	The floor plan supplied with the problem statement, page two of the PDF, covering both buildings.
Test data	Records created to demonstrate the system. Every one of them carries a flag and a visible marker in the interface.

TABLE 1 *Terms used throughout. Where a term means something narrower in the code, the code name is given in capitals.*

SECTION 2

The problem as the campus experiences it

Last year a group of students missed a minor examination. The arrangements had changed and the change did not reach them. It is tempting to file that under communication, and to suppose that a group chat would have fixed it. It would not have. The arrangements changed because a room was needed somewhere else at the same hour, and nobody could see, in one place, which rooms were promised to whom. The information failure was the visible symptom. The allocation failure came first.

That order matters, because it decides what to build. A notice board with a good search box would have told students the wrong room more efficiently.

2.1 What a working system has to do

Four things, and the fourth is the one the problem statement names last but weighs most heavily:

1. Tell a member of faculty, before they walk across the campus, which rooms are free for a stated number of students at a stated time, and what each of those rooms can hold.
2. Take a booking without ever permitting two activities to occupy one room at one time, and when a clash is unavoidable, offer real alternatives rather than a refusal.
3. Carry changes to the people affected by them, automatically, at the moment the change is made.
4. Keep a record of who promised what and who changed it, so that a disagreement can be settled from evidence rather than from memory.

2.2 The buildings decide a great deal

The Institute of Engineering and Technology is a rectangular block of three floors. Rooms sit in two rows on either side of a corridor, and a single staircase bisects the two rows at the centre of the building, immediately inside the only entrance. There is no lift. The ground floor holds the library, the teachers' office, the placement cell and the director's office; the first floor holds four rooms that the statement describes as laboratories and eight rooms in total; the second floor holds the seminar hall and six classrooms.

Deeksha Bhawan stands about a hundred metres away. It is a ring of rooms around a core containing a washroom and a drinking-water point, with a single staircase beside the south entrance. There are fourteen rooms across its two floors, numbered LH01 to LH14 on the plan, and they are not interchangeable. The two long rooms on each floor run the full depth of the plan and will seat a full section. The rest are compact.

Five consequences follow, and each of them is a requirement rather than a description.

What the plan shows	What it forces the software to do
One staircase and one entrance per building, and no lift in either	Changing floor costs time and creates crowding. Two activities for the same section must never be scheduled back to back across a floor change without a buffer.
Deeksha Bhawan is a hundred metres from IET, across open ground	A recommendation that moves a class to the other building is a real cost to real people. It must be priced, displayed, and refused when the gap is too small.
Only two spaces on the whole campus can seat a full section with examination spacing	Examination placement is a scarcity problem, not a booking problem. The system has to propose split sittings, and to plan them days ahead.

What the plan shows	What it forces the software to do
Offices sit between teaching spaces on the IET ground floor	Offices must be excluded from availability, or the search will offer the director's office as a spare room.
Room numbering is spoken aloud: "IET nineteen", "LH eight"	The system uses the campus's own names as identifiers. Inventing IET-FF-19 would break the one thing the software must match: the way people talk.

TABLE 2 *Structural facts from the blueprint, and the requirement each one produces.*

2.3 The arithmetic of scarcity

There are thirty bookable rooms and eight teaching periods in a day, which gives two hundred and forty room-periods of supply. There are thirty-two sections: two each in computer science and in artificial intelligence and machine learning, and one each in artificial intelligence and data science, electronics, mechanical and information technology, across four years. Every section is seventy-five strong, because the branches that are divided in two have one hundred and fifty students and the branches that are not have seventy-five.

At six periods a day, those sections need a hundred and ninety-two room-periods, or eighty per cent of the supply. At seven periods, which is what an examination week looks like, they need two hundred and twenty-four, or ninety-three per cent. Two observations follow immediately, and both are uncomfortable.

First, a single period can host at most thirty sections, because there are thirty rooms. The thirty-first and thirty-second sections must either be idle or be combined, and that constraint is a property of the building rather than of any timetable. Second, demand does not spread evenly. Morning periods and the period after lunch carry most of the pressure. If two thirds of a day's activity wants the same three periods, those periods are oversubscribed by half again, and the queue is what faculty experience as "there is never a room free".

The examination case is sharper still. With alternate-seat spacing, a seventy-five-seat classroom offers thirty-seven usable seats. A section of seventy-five students sitting an examination therefore cannot use a classroom at all. It needs the seminar hall, which seats two hundred, or the library, which seats about a hundred and eighty, or it must be split across two rooms. There are exactly two rooms on this campus that qualify. That is not an oversight by anyone; it is arithmetic, and until it is written down it tends to be discovered at the door of the examination hall.

THE SENTENCE THAT ORIENTS THE WHOLE DESIGN

This is not a calendar application. It is an allocation system operating on a scarce, shared, physical resource, and the interesting engineering is in the allocation and in the record of it, not in the grid that displays it.

KEY The information failure students experienced was the symptom. The allocation failure came first, and it is arithmetic: thirty rooms, thirty-two sections, and only two spaces that can hold a spaced examination.

SECTION 3

The proposal we began with, and the four defects in it

3.1 The proposal, stated fairly

Our first sketch was short, and it is worth recording exactly as we wrote it, because the idea in it is not wrong so much as under-specified:

1. Create a bucket for each room, named as the campus names it, IET14, IET15, IET16, LH04 and so on, and record the maximum number of students each can hold.
2. Let a member of faculty enter a headcount together with the class details, subject, year, branch and time, and in return show the rooms that are empty and large enough.
3. Allow a room to be marked free, after which it is available to whoever needs it.
4. Where attendance is low, or a class is cancelled, mark the class as having none present, and treat the room as free.

3.2 What was right about it

Three things, and we kept all three. The room register with a capacity is the correct foundation, and it survives into the final design as a static table that no booking ever modifies. Filtering by headcount is the correct primary workflow; it is what a member of faculty actually wants to do. And the instinct that rooms should return to a pool rather than be consumed is right, even though the mechanism we first proposed for it was not.

3.3 The four defects

The proposal treated a room as a switch with two positions. That single decision produces four separate failures, and they compound.

Defect one: a switch cannot be asked about the future

A flag reading OCCUPIED answers the question "is somebody in this room now?" A member of faculty needs to know whether a room will be free tomorrow between ten and eleven, and for how long after that. Those are different questions, and a flag cannot be made to answer the second one without storing a time as well, at which point it is no longer a flag.

Defect two: marking a room free destroys the record

If freeing a room means clearing or deleting the entry, the history goes with it. The problem statement asks, in its seventh requirement, for a history of important bookings and changes so that misunderstandings can be resolved. The mechanism we proposed would have deleted precisely the evidence that requirement asks for, one room at a time.

Defect three: a room released in the middle of a period is not usable

Suppose a class begins at ten and the room is marked free at twenty past ten. Thirty-five minutes remain in the period. No member of faculty can use them. Sixty students are still in the room with their bags, a projector is in use, and a second class cannot start for half a period and then stop. The release frees a room on the screen and nowhere else.

Defect four: attendance is not an allocation signal

This is the defect we consider serious. Marking a class as having nobody present does two jobs at once, and they contradict each other. As an attendance record it says that no student attended. As an allocation signal it says the room can be given away. A class that is genuinely cancelled and a class that every student skipped become the same row, which is the exact ambiguity that made last

year's examination notice impossible to correct. It also creates an incentive nobody wants: release a room for a better slot by marking students absent.

3.4 A worked counter-example

Take a Wednesday. CSE second year section A has a database class in IET20 from ten to ten fifty-five. At twenty past ten the class is running; the projector is on and seventy-two students are seated. Under the original proposal the system is asked for a room for another section from half past ten until half past eleven, and the answer depends on the flag.

Moment	Under the original proposal	Under the corrected model
10:00	IET20 marked occupied	A reservation exists for 10:00 to 10:55
10:20	Faculty marks the class as having none present, so the room is released	Nothing changes: the reservation covers the whole interval
10:25	The system offers IET20 to another section for 10:30 to 11:30, which is eighty minutes of a room that is in use for thirty-five more of them	IET20 is reported as busy until 10:55, and free from 10:55; alternatives are offered for the whole window
10:30	Two classes occupy one room; the second group stands in the corridor	The second section books IET18, which is free and adjacent
11:00	The attendance record for the first class shows nobody present, contradicting seventy-two students who were examined in that room	The cancellation, if there was one, is recorded as a cancellation with a reason, and attendance is a separate record held by the department

TABLE 3 *The same morning under both models. The failure at 10:25 is not an edge case; it is the ordinary consequence of releasing a room in the middle of a period.*

KEY Committed live reservations are what the database protects, not the physical room. A cancelled row therefore releases its interval while remaining part of the record, which is how the same design serves both correctness and accountability.

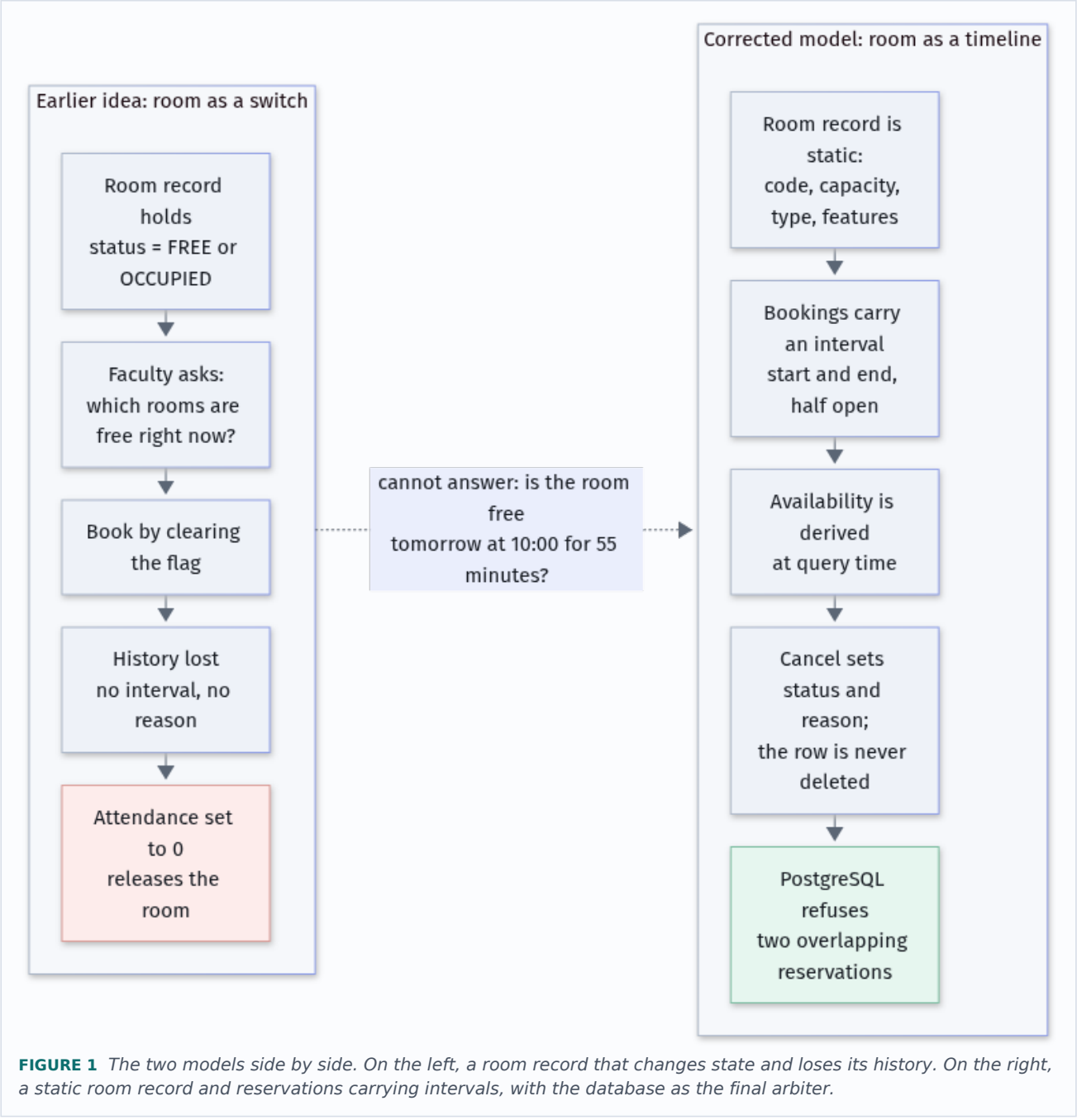


FIGURE 1 *The two models side by side. On the left, a room record that changes state and loses its history. On the right, a static room record and reservations carrying intervals, with the database as the final arbiter.*

One further consequence deserves mention because it took us a while to see it. Under the original proposal, the room register was going to be the thing that changed. Every booking would edit it, every cancellation would edit it back, and after a term of that, the register would hold the current state and nothing else. A register that is edited by bookings cannot also be the reference table against which bookings are validated. Splitting the two, a static room table and an append-only reservation table, is what makes the rest of the design possible.

PART II

The corrected design

Rooms as timelines, three constraints, the three roles, the schema and the data flow. This part is the contract for implementation.

— 4 The corrected model: rooms are timelines, not switches
— 5 The three roles and how each of them works
— 6 Database design
— 7 Data flow

SECTION 4

The corrected model: rooms are timelines, not switches

4.1 The principle in one sentence

A room is a fixed description of a physical space. A reservation is a claim on that space for a stated interval. Availability, the only question that changes from minute to minute, is computed from the claims and is never stored anywhere. Every other decision in this document follows from that split.

4.2 Availability, derived on every query

Three things can occupy a room: a reservation, a recurring class in the official timetable, and a closure for repairs, holidays or an institutional event. The system takes the union of those three sets of intervals for the room in question and asks whether the requested window intersects it. Nothing else is consulted, and nothing is cached across requests.

The consequence worth noticing is that free time is a gap, which is why a cancellation restores availability without any additional work. When the status of a reservation leaves the set that counts as occupying, the gap it had filled becomes a gap again.



FIGURE 2 *One morning on three rooms, as the system reconstructs it. The centre block of IET20 is a cancelled class: the row still exists, and only the interval returned to the pool.*

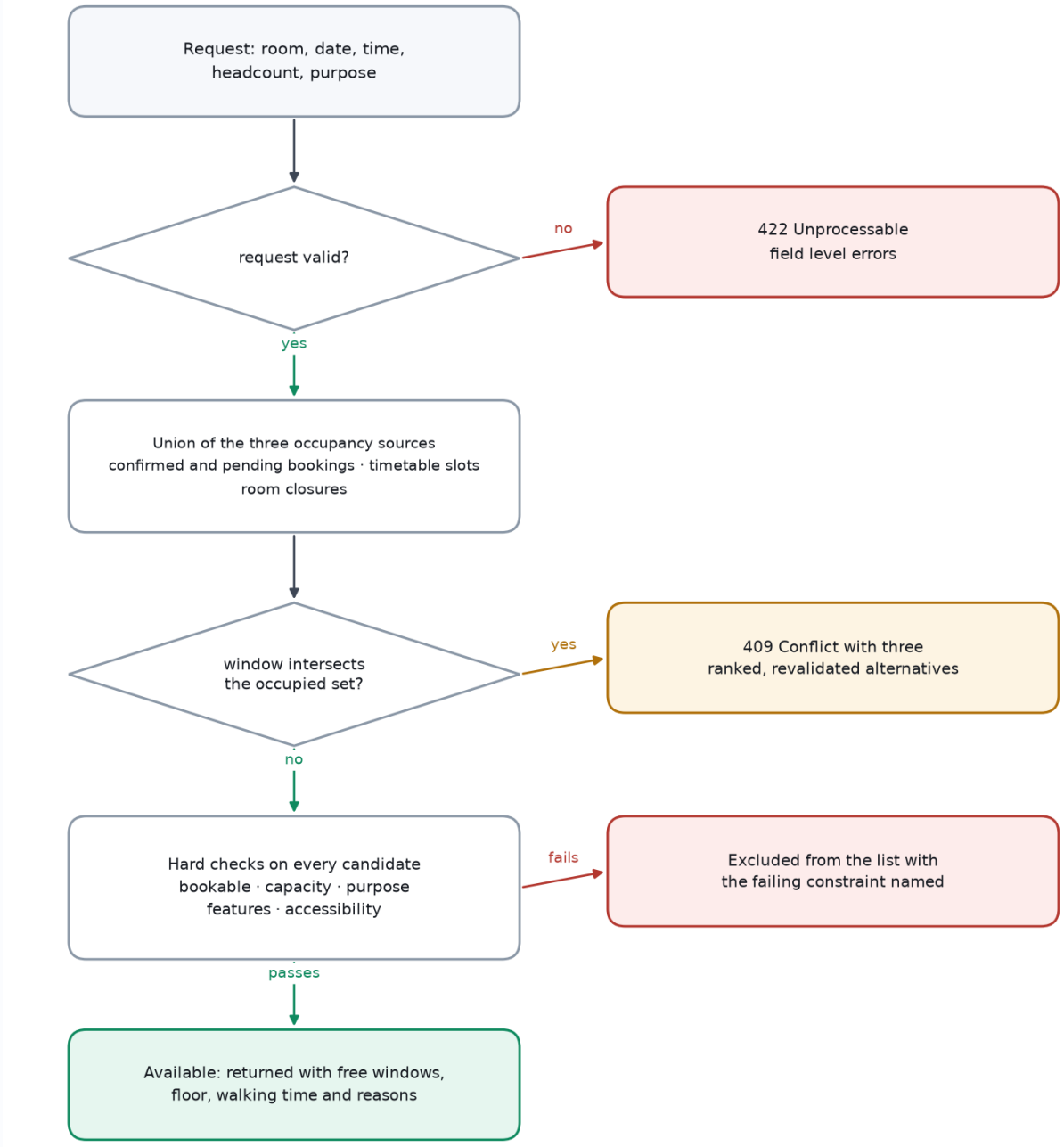


FIGURE 3 *The availability query as a decision path. The three occupancy sources are united first, so a window that intersects any of them never reaches the hard checks.*

```
-- One query answers the availability question. This is the whole of it.
SELECT r.code, r.capacity, f.level AS floor, b.code AS building
FROM room r
JOIN floor f ON f.id = r.floor_id
JOIN building b ON b.id = f.building_id
WHERE r.bookable
AND r.capacity >= :headcount
AND r.id NOT IN (
SELECT room_id FROM booking
WHERE status IN ('CONFIRMED', 'PENDING_APPROVAL')
AND time_range && tstzrange(:start, :end, '[)')
UNION
SELECT room_id FROM room_closure
WHERE time_range && tstzrange(:start, :end, '[)')
UNION
SELECT room_id FROM timetable_slot
WHERE weekday = :weekday AND period && :periods)
ORDER BY (b.code = 'IET') DESC, r.capacity ASC; -- prefer IET, then tightest fit
```

CODE *Listing 1. Availability query. Ordering by capacity ascending keeps the large rooms free for the events that need them.*

4.3 Three constraints, because a room is not the only thing that can be double-booked

A booking can collide with three different things, and only the first of them is obvious. A room can be promised twice. A section can be sent to two rooms at once, which is what students experience as "the class was moved and nobody told us". A member of faculty can be scheduled in two rooms at once, which is usually discovered when they fail to appear.

All three are prevented in the database, not in the application. The mechanism is PostgreSQL's exclusion constraint over a range type, which makes a conflicting row impossible to insert rather than merely unlikely.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
-- 1. no room carries two live reservations for the same interval
ALTER TABLE booking ADD CONSTRAINT booking_room_no_overlap
EXCLUDE USING gist (room_id WITH =, time_range WITH &&)
WHERE (status IN ('CONFIRMED', 'PENDING_APPROVAL'));
-- 2. no section is expected in two rooms at the same time
ALTER TABLE booking ADD CONSTRAINT booking_section_no_overlap
EXCLUDE USING gist (section_id WITH =, time_range WITH &&)
WHERE (status IN ('CONFIRMED', 'PENDING_APPROVAL'));
-- 3. no member of faculty is expected in two rooms at the same time
ALTER TABLE booking ADD CONSTRAINT booking_faculty_no_overlap
EXCLUDE USING gist (requested_by WITH =, time_range WITH &&)
WHERE (status IN ('CONFIRMED', 'PENDING_APPROVAL'));
```

CODE *Listing 2. The three constraints. Together they are about twenty lines, and they remove an entire class of failure that application code cannot remove.*

The third column of each constraint, the predicate, is what makes cancellation work. Only reservations that are confirmed or awaiting approval participate. A cancelled row stays in the table, keeps its reason, keeps its place in the audit trail, and stops occupying the interval.

4.4 Cancel, never delete

Every state change is a status change with a reason attached. Nothing is deleted, ever, and the interface never offers the option. A cancelled reservation is the record that a promise was made and then withdrawn, which is exactly what a dispute needs.

Status	Meaning	Occupies the interval	Who can set it
PENDING_APPROVAL	Requested for a room that needs authorisation: the seminar hall, the library, or an event above a hundred people	Yes	Faculty, on submission
CONFIRMED	Promised to the requester	Yes	Faculty, or an administrator on approval
CANCELLED	Withdrawn, with a mandatory reason	No	The requester before the start, or an administrator
REJECTED	Refused at approval, with a reason	No	An administrator
COMPLETED	The interval has passed and the record is closed	No	The system, after the end
DISPLACED	Overridden by an administrator for an examination, with notice	No	An administrator only

TABLE 4 *Reservation states. Pending requests hold their interval, otherwise three requests could all be approved for one room at the same hour.*

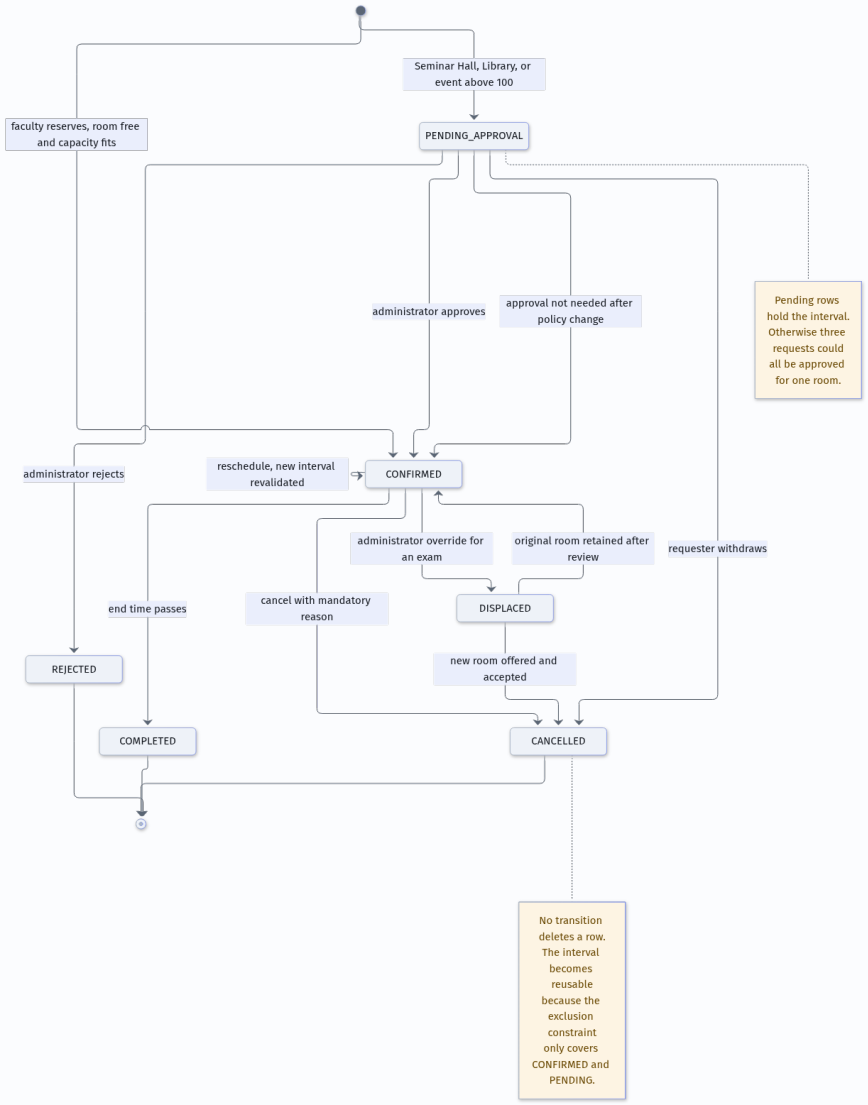


FIGURE 4 *The life of a reservation. There is no transition that deletes a row, and the cancelled state is what returns an interval to the pool.*

4.5 Attendance belongs to a different system

We removed attendance from the allocation model entirely, and the removal is deliberate enough to state as a design rule. A class that is cancelled is recorded as a cancellation, with a reason, by the member of faculty responsible. Attendance, meaning who was present, is an academic record that belongs with the department, is contestable by students, and has no route into the booking table.

Two mechanisms replace what the attendance flag was meant to achieve, and they do it better. The first is planned cancellation: a class that will not run tomorrow is cancelled today, which releases a whole period, notifies the affected sections, and lets another class take the slot in good time. The second is early release, where a member of faculty shortens a reservation while it is in progress, so that the remainder of the interval returns to the pool. Both write an audit record. Neither touches attendance, and neither can be triggered by it.

> **RULE**
> Attendance may describe what happened in a room. It may never determine who is allowed to use one. Where the two seem to conflict, the reservation wins and the conflict is recorded as a report.

4.6 What a member of faculty sees when a room is taken

A conflict is not an error message. It is a question that has already been answered three ways. The response carries the reason for the refusal and a ranked list of substitutes, each with the reasoning attached, so the person at the keyboard can decide in seconds rather than starting again.

```
HTTP/1.1 409 Conflict
{
"code": "ROOM_ALREADY_BOOKED",
"message": "IET19 is booked from 10:00 to 10:55 for a practical session (CSE-2A).",
"alternatives": [
{ "room": "IET18", "start": "10:00", "end": "10:55", "score": 94,
"reasons": ["adjacent room on the same floor", "seats 75 against 72 requested"] },
{ "room": "IET24", "start": "10:00", "end": "10:55", "score": 81,
"reasons": ["one floor up, about forty-five seconds on the stair"] },
{ "room": "LH08", "start": "10:20", "end": "11:15", "score": 63,
"reasons": ["Deeksha Bhawan, one hundred metres away", "start shifted by 20 minutes"] }
]
}
```

CODE *Listing 3. A conflict response. The third alternative is allowed to move the window, which is what turns a refusal into a decision someone can act on.*

SECTION 5

The three roles and how each of them works

Three kinds of people use the system, and they want incompatible things from it. A member of faculty wants to secure a room quickly. A student wants to be certain about where to be at nine in the morning. An administrator wants nothing to happen that they did not sanction, and to be able to explain afterwards what did. The design gives each of them a different front door to the same record.

Action	Student	Faculty	Administrator
View the weekly schedule for a section	Yes	Yes	Yes
Search room availability	Yes, read only	Yes	Yes
Reserve a classroom or laboratory, up to seventy-five students	No	Yes	Yes
Reserve the seminar hall, the library, or an event above a hundred	No	Requests approval	Approves
Reschedule or cancel a reservation	No	Own reservations only	Any, with a reason
Publish a notice to a section	No	Own sections only	Yes
Publish a campus or building-wide notice	No	No	Yes
Comment on a reservation, or report a problem	Yes	Yes	Yes
Resolve a report, correct room data, change policy	No	No	Yes
Read the audit trail	No	Own actions and affected rooms	All

TABLE 5 *Permission matrix. Every cell is asserted in a test, including the negatives, which is what stops an interface change from becoming a privilege leak.*

5.1 A member of faculty

The journey is four screens and it is the part of the system that has to feel fast. The person enters a headcount and the class details; the system answers with rooms that fit, each showing its floor, its walking time from the current building, and the free windows inside the requested period. Selecting a room produces a confirmation which immediately appears on the schedule of the affected sections.

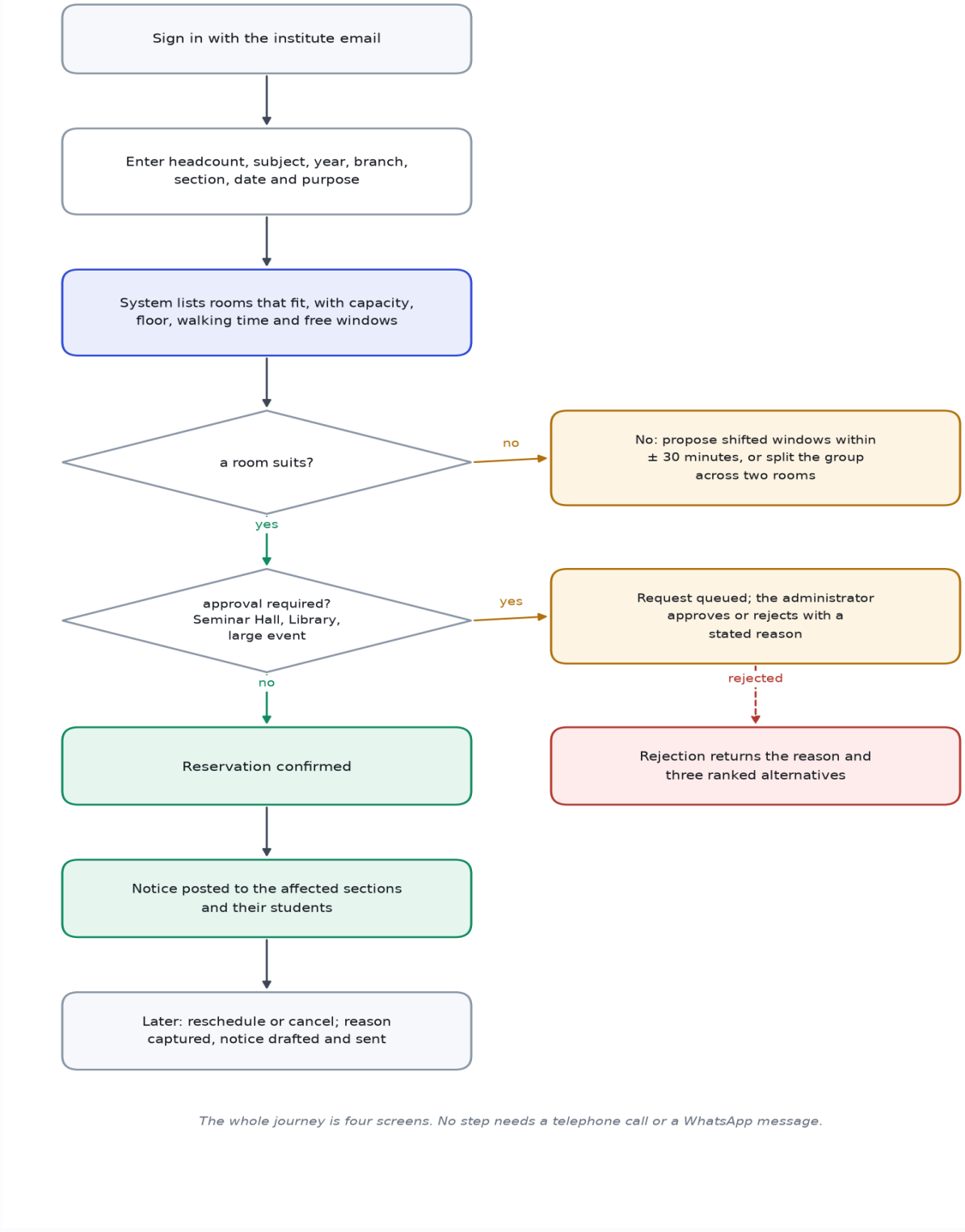


FIGURE 5 *The faculty journey, including the two branches that make the system useful: no room fits, and approval is required.*

Two details in that flow do more work than their size suggests. First, when no room fits, the system does not return an empty list; it looks for windows within half an hour of the request, and it will propose a split across two rooms when the headcount exceeds what any single room can take for an examination. Second, the alternatives are ranked by a scoring function whose weights are visible in the administrative settings, so a member of faculty who disagrees with a suggestion can see why it was made.

5.2 A student

Students read. That is the whole of their write access to the schedule, and the restriction is enforced in the policy layer rather than by hiding a button. What they can do is speak: a comment attached to a reservation, visible to its owner and to an administrator, and a structured report when something is visibly wrong.

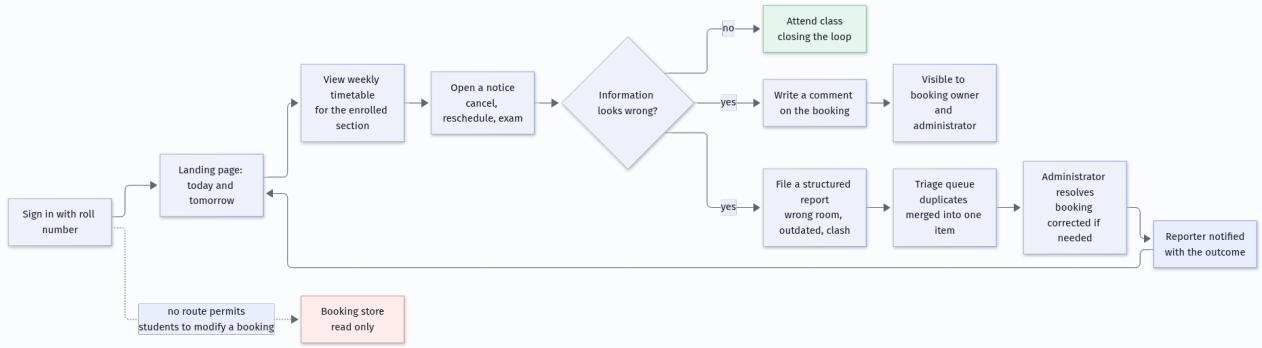


FIGURE 6 *The student journey. The lane on the right exists so that a wrong room reaches somebody who can correct it, without any student being able to edit the schedule.*

Reports are structured rather than free text, because structure is what makes them triageable: the wrong room, a cancelled class still showing, a clash with another class, outdated information, or something else. Duplicate reports on the same reservation are merged, so twelve students reporting the same wrong room produce one item with a count of twelve, which is precisely the signal an administrator needs. When the item is resolved, the reporters are told the outcome. A channel that never answers trains people to stop using it.

5.3 An administrator

The administrator's workload is concentrated in five screens: confirming room data, clearing the approval queue, resolving reports, changing policy, and reading the audit trail. Of these, the first is the smallest and the most important. Five facts about the campus are inferred rather than stated, and each of them is presented for confirmation with its source and its confidence so that it can be corrected in one click.

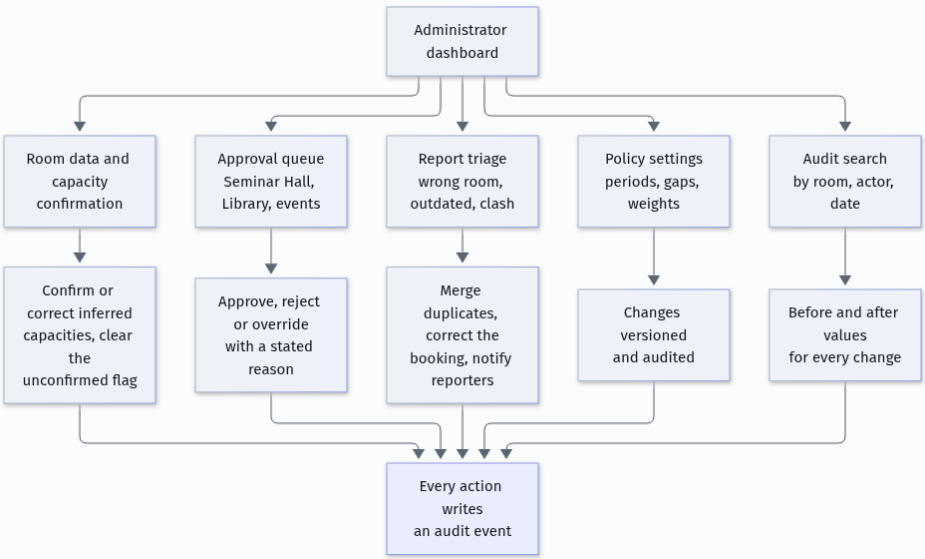


FIGURE 7 *The administrator's five surfaces, all of which write to the same audit trail.*

5.4 How the restrictions are actually enforced

The matrix in the table above is not documentation of scattered conditionals. It is implemented as one function, called by every route handler before any work is done, which returns not only whether an action is permitted but why it was refused and whether it needs approval instead.

```
// One decision point for the whole application.
export function can(actor: Actor, action: Action, resource?: Resource): Decision
// { allowed: boolean, reason?: string, requiresApproval?: boolean }
// Every route begins the same way.
const decision = can(actor, 'booking:cancel', booking);
if (!decision.allowed) {
return problem(403, decision.reason); // the reason is shown to the user
}
```

CODE *Listing 4. Authority is decided once. A parameterised test walks the matrix and asserts every cell, so a new route cannot quietly widen a role.*

We test the negative cells specifically, by signing in as a student and replaying a faculty request with the student's own session. The expected answer is a refusal with a reason, and the test exists because a review will eventually ask what happens when somebody edits the page in a browser console. Hiding a control is a convenience; the refusal at the route is the security.

SECTION 6

Database design

6.1 Five rules that decide the schema

1. **Instants, not wall-clock strings.** Every boundary is a timestamp with time zone. The campus lives in one zone, but that zone is five and a half hours from Greenwich, and a half-hour offset breaks any code that assumes whole hours.
2. **Half-open intervals.** A reservation occupies [start, end). One ending at 11:00 and one beginning at 11:00 do not collide, and that single decision removes the most common complaint about booking systems.
3. **Snap input to five minutes, compare exactly.** The grid gives tidy data entry; the interval comparison gives correctness. Practical sessions start at odd times, and the model must allow that.
4. **Nothing is deleted.** Cancellation is a status. The table is a record of promises, and a promise withdrawn is still part of the record.
5. **Identifiers come from the campus.** IET20, IET32, LH08. Not a generated string, not a renumbered code. The system must be usable by somebody saying "put them in IET nineteen".

6.2 The entities

Entity	Why it exists	Fields that carry decisions
BUILDING	The two buildings differ in ways the algorithm must price	code, distance_to_iet_m, has_lift, entry_gate
FLOOR	Floor changes cost time, so floor must be queryable	building_id, level
ROOM	The resource being allocated; static, never edited by a booking	code, floor_id, room_type, capacity, exam_capacity, bookable, accessibility, zone, permitted_uses, unconfirmed
ROOM_CLOSURE	Repairs, holidays and institutional events make a room unusable	room_id, time_range, reason
SECTION	The demand unit; capacity is checked against section strength	code, branch, year, strength
STUDENT	Links a person to a section for the schedule view	roll_number, section_id
FACULTY	Links a person to the reservations they own	email, department
COURSE	Ties a reservation to a curriculum item and its room needs	code, title, branch, year, required_features
TIMETABLE_SLOT	The official recurring classes, which are the background load	course_id, section_id, room_id, weekday, period, valid_from, valid_to
BOOKING	The core fact: a claim on a room for an interval	room_id, section_id, faculty_id, time_range, purpose, headcount, status, reason, is_test, version, idempotency_key
BOOKING_SERIES	Weekly classes must be created and cancelled as one action	recurrence_rule, parent_booking_id

Entity	Why it exists	Fields that carry decisions
BOOKING_EVENT	The human-readable life of one reservation	booking_id, event_type, actor, at, detail
AUDIT_EVENT	The tamper-evident trail across the whole system	actor, action, entity, entity_id, before_state, after_state, request_id, at
ANNOUNCEMENT	Carries changes to the people affected	scope, scope_id, severity, title, body, publish_at, expires_at, supersedes_booking_id, version
REPORT	Lets students and faculty challenge the record without editing it	booking_id, reporter, kind, status, thread
NOTIFICATION	Outbox rows so a mail failure cannot undo a booking	user_id, channel, payload, status, attempts, sent_at
ATTENDANCE	Deliberately separate from allocation	student_id, course_id, date, state

TABLE 6 *Seventeen entities. Anything that could not be justified in one sentence was left out. There is no message broker, no cache table and no catch-all settings blob; the policy values in section 12.3 have their own table with a history.*

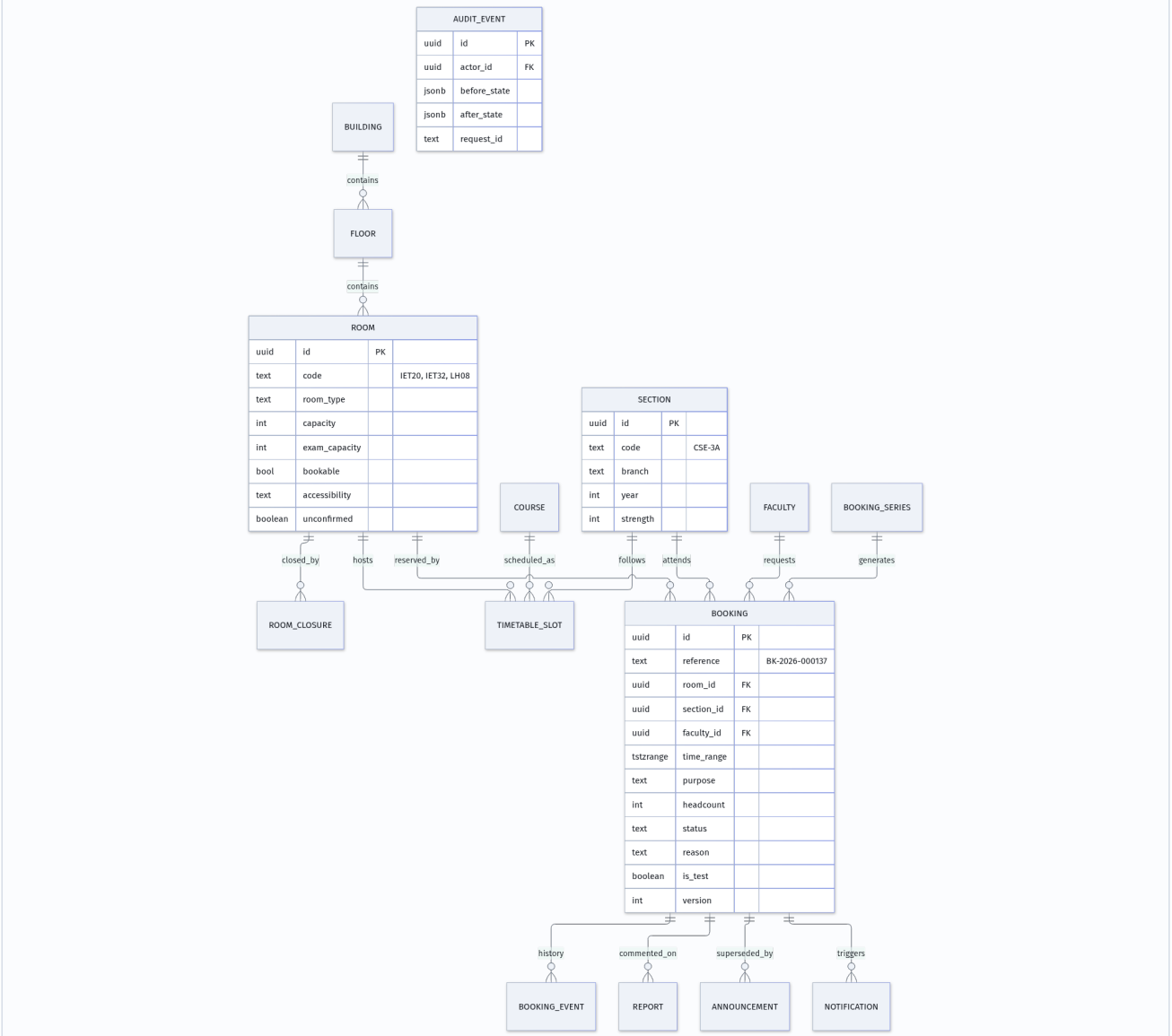


FIGURE 8 *The entity relationship model. Booking sits at the centre because every correctness rule is a rule about booking.time_range.*

6.3 Indexes and the performance envelope

Operation	Target at the ninety-fifth percentile	How it is achieved
Availability for one day, both buildings	under 80 ms	GiST index on the interval, composite index on room and start time, one query and no per-room loop
Creating a reservation	under 150 ms	one transaction, three inserts, and no external call on the critical path
Generating alternatives, twelve rooms across five shifted windows	under 120 ms	one batched query for all candidates, scoring in memory
A section's weekly schedule	under 200 ms	server-rendered page with a tagged cache invalidated when a notice is published
Natural-language search end to end	under 2.5 s	streamed response, one tool call, cached system prompt, skeleton in the interface

TABLE 7 *Performance budget. The numbers are targets for the load test in section 11, not measurements from a production system.*

```
CREATE INDEX booking_room_start_idx ON booking (room_id, start_at DESC);
CREATE INDEX booking_range_gist ON booking USING gist (time_range);
CREATE INDEX booking_section_idx ON booking (section_id, start_at DESC);
CREATE INDEX booking_live_idx ON booking (status) WHERE status IN
('CONFIRMED','PENDING_APPROVAL'); -- partial index: only live rows matter
```

CODE *Listing 5. Indexes. The partial index on live statuses is small and is the one the availability query uses most.*

6.4 Reference data is a file, not a habit

The room register is stored once as a versioned data file in the repository, and the seed command reads it. Rooms are never typed into a component, a fixture and a migration separately, which is how a room ends up with a capacity of seventy-five in the interface and two hundred in the database. The complete register is printed in Appendix A; the summary below is enough to see the shape of the problem.

Building and floor	Bookable rooms	Capacity band	Notes
IET, ground floor	1	180	Library, dual use as an examination hall, approval required
IET, first floor	8	75 each	Four rooms described as laboratories, four classrooms
IET, second floor	7	200 and 75	Seminar hall IET32 seats two hundred; six classrooms
Deeksha Bhawan, ground floor	7	45 to 80	LH01 and LH02 are the long rooms
Deeksha Bhawan, first floor	7	45 to 80	LH08 and LH09 are the long rooms
Total	30 bookable of 34 recorded		Four rooms are offices or a service space and are not bookable

TABLE 8 *The register in summary. Ten of the thirty rooms cannot seat a full section, and fourteen can seat a section but not an examination sitting.*

6.5 Where the data comes from, and where it does not

The blueprint is authoritative for which rooms exist, where they are and how they are numbered. The problem statement is authoritative for the capacities it states. Five facts are in neither, and rather than guess silently we recorded them as assumptions with a flag on the row.

The disagreement	What we did	Why
The statement calls the four first-floor north rooms laboratories for computer science, mechanical, electronics and information technology; the plan draws four unlabelled rooms	Recorded as laboratories, mapped to those branches, flagged as unconfirmed	One configuration row changes them to classrooms if the organisers clarify, and the flag is visible in the interface
The statement lists five second-floor classrooms; the plan shows six, including IET23	Kept as a bookable room, flagged as unconfirmed	The plan is specific where the statement is a summary, and dropping a usable room costs more than keeping one that may not exist
The statement says around ten to twelve rooms in Deeksha Bhawan; the plan shows fourteen, LH01 to LH14	Kept all fourteen, flagged LH13 and LH14	The word around licenses the plan, and the flag preserves honesty
No capacity is given for any room in Deeksha Bhawan	Inferred from relative plan area: the long rooms at eighty, the medium rooms at fifty-five, the compact rooms at forty-five	Stored as editable values and marked assumed until an administrator confirms them
The library is used for examinations sometimes	Bookable for examinations and events, with an approval gate and an exclusion for library hours	Modelled as policy rather than as a hard denial, because the policy will change

TABLE 9 *The reconciliation ledger. Stating these openly is stronger than resolving them quietly, because each one is a decision the panel can check.*

SECTION 7

Data flow

The diagrams in this section follow a conventional decomposition: a context diagram to fix the boundary, a first-level diagram to show the processes and the stores, a second-level diagram for the one process where correctness is decided, and three sequence diagrams for the interactions that matter most.

7.1 Context

Three kinds of user, two supporting systems and one store of record. The language model appears on the boundary deliberately, and with an arrow in each direction that is not symmetric: it may read results and return a structured intent, and it has no path that writes.

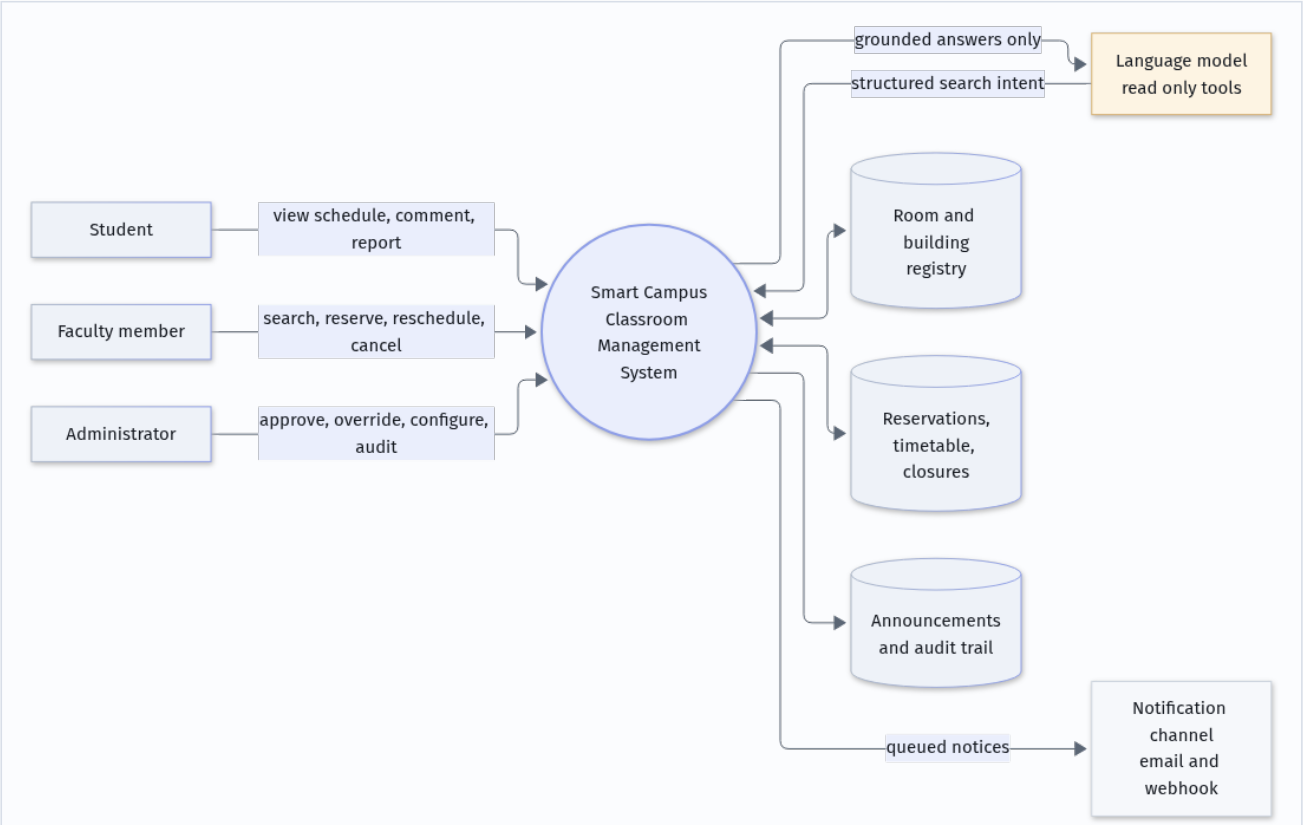


FIGURE 9 *Context diagram. The asymmetry of the two arrows to the language model is the design decision: propose, never commit.*

7.2 First level

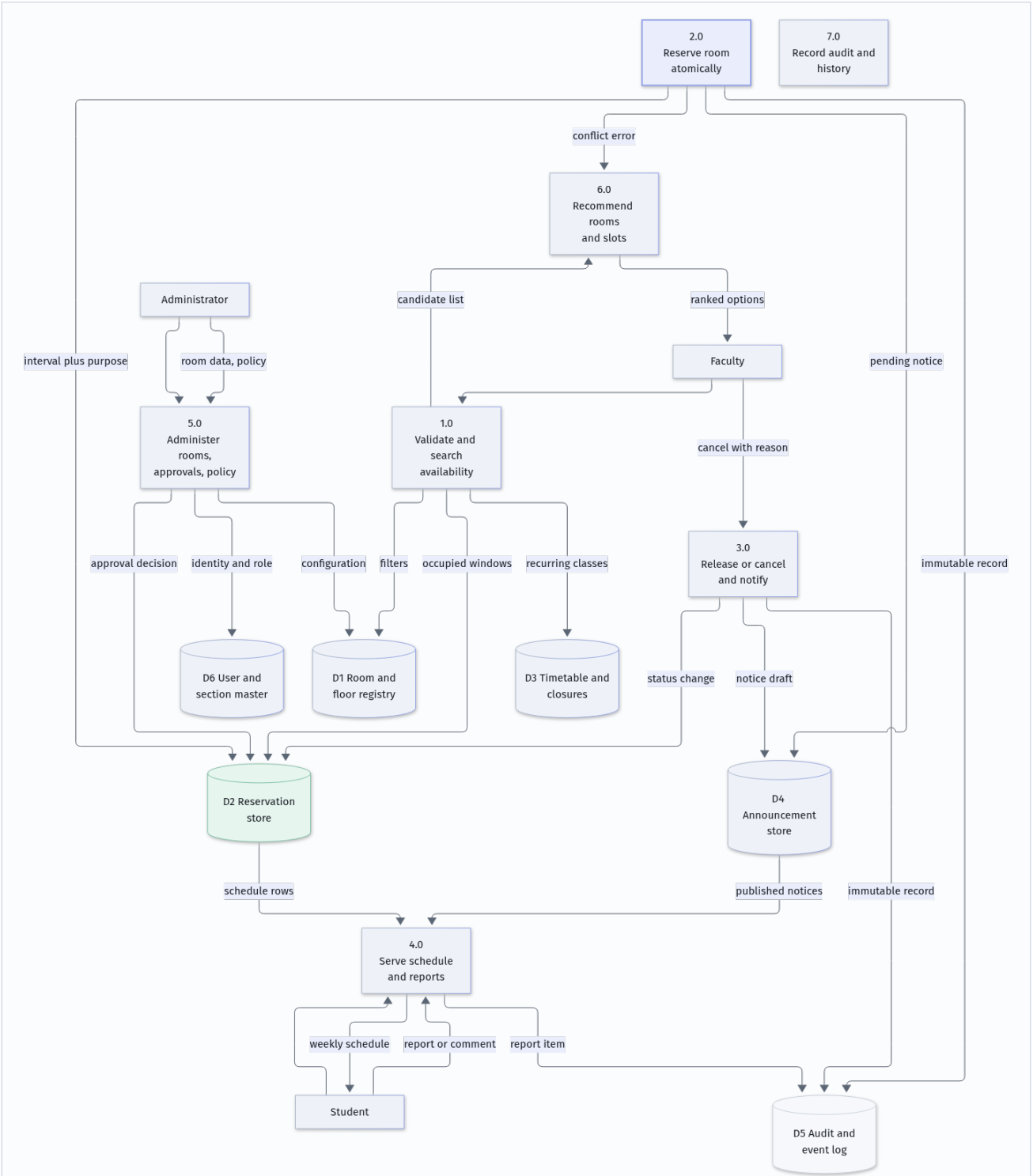


FIGURE 10 *First-level data flow. Process 2.0 is where every conflict is resolved, and it is expanded in figure 11.*

The data dictionary for the flows that carry decisions:

Flow	Composition	Where it is validated
Availability filters	building, floor, room code, date, start, duration, headcount, purpose, accessibility	Schema validation at the route, before any query
Occupied windows	room, interval, source (reservation, timetable, closure)	Produced by the union query in Listing 1

Flow	Composition	Where it is validated
Candidate list	room, capacity, floor, walking time, free windows, score, reasons	Re-validated against the store immediately before display
Reservation request	room, interval, purpose, headcount, section, faculty, idempotency key	Authorisation, then policy, then capacity, then the exclusion constraint
Conflict report	the occupying reservation and three ranked alternatives	Alternatives are re-queried after the constraint fires, never reused from a cache
Notice draft	scope, scope id, severity, body, supersedes	Generated inside the same transaction as the status change
Audit record	actor, action, entity, before, after, request id	Written by the same transaction, never by the client

TABLE 10 *Flows that carry decisions, and the point at which each is checked.*

7.3 Second level: the booking transaction

This is the one process where the system is either correct or not, so it is worth following in full. Note where the database, rather than the code, has the last word.

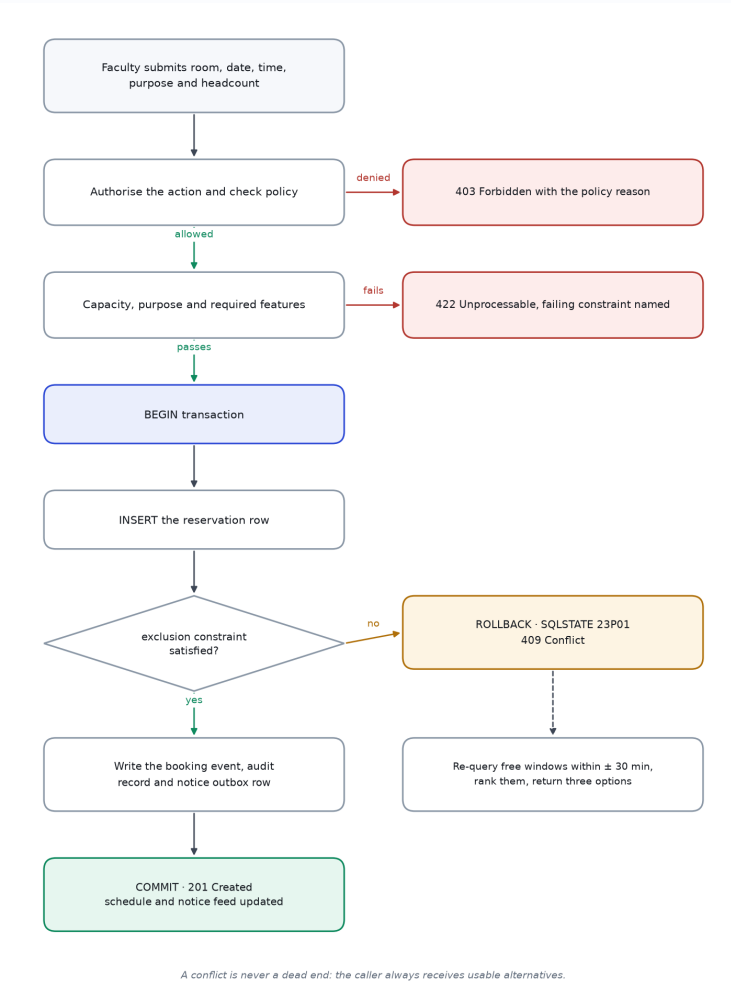


FIGURE 11 *Second-level flow for reserving a room. The rollback branch is not an error path; it is the normal path for the second of two simultaneous requests.*

7.4 Two members of faculty, one room, one second

The sequence below is the argument for putting the rule in the database. Both requests query availability, both are told the room is free, and both are correct at the moment they ask. Only the insert can separate them.

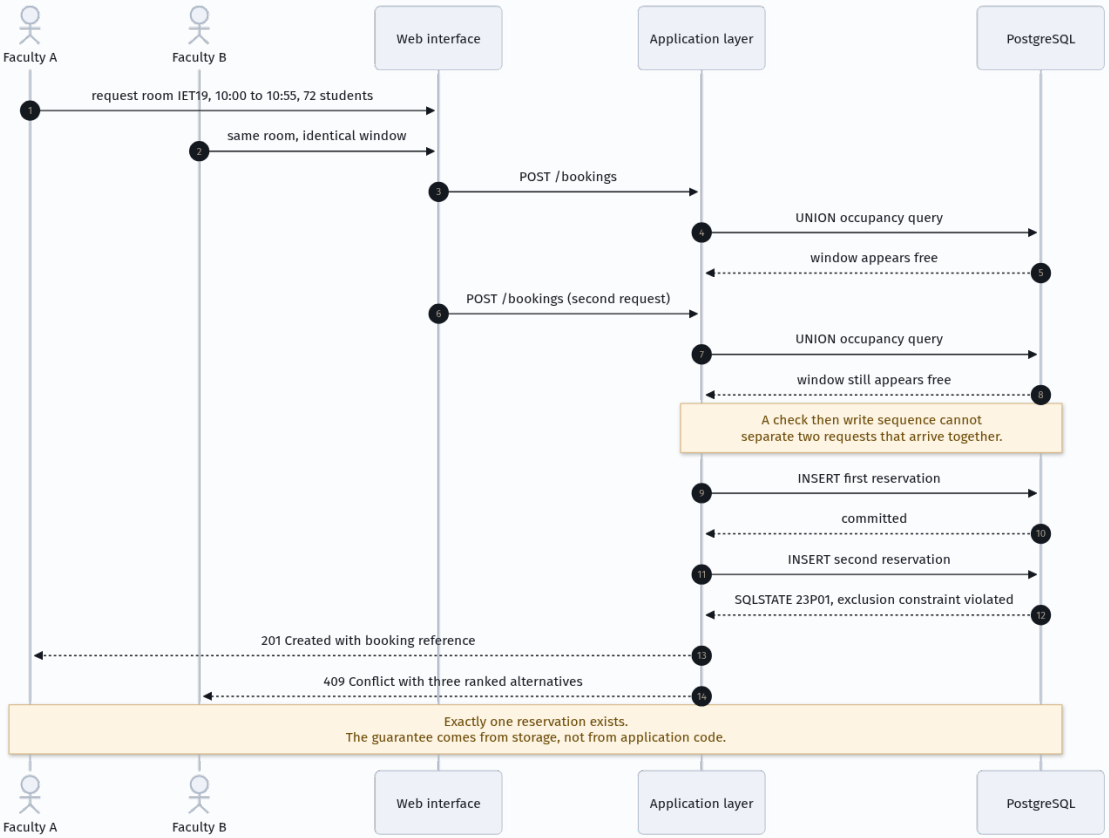


FIGURE 12 *Two concurrent reservations. The second receives a refusal with alternatives, and exactly one row exists afterwards.*

7.5 Cancellation and the fan-out to students

Cancelling is one action for the person doing it and five records in the database. The notice is written inside the same transaction as the status change, which is what makes it impossible to cancel a class silently.

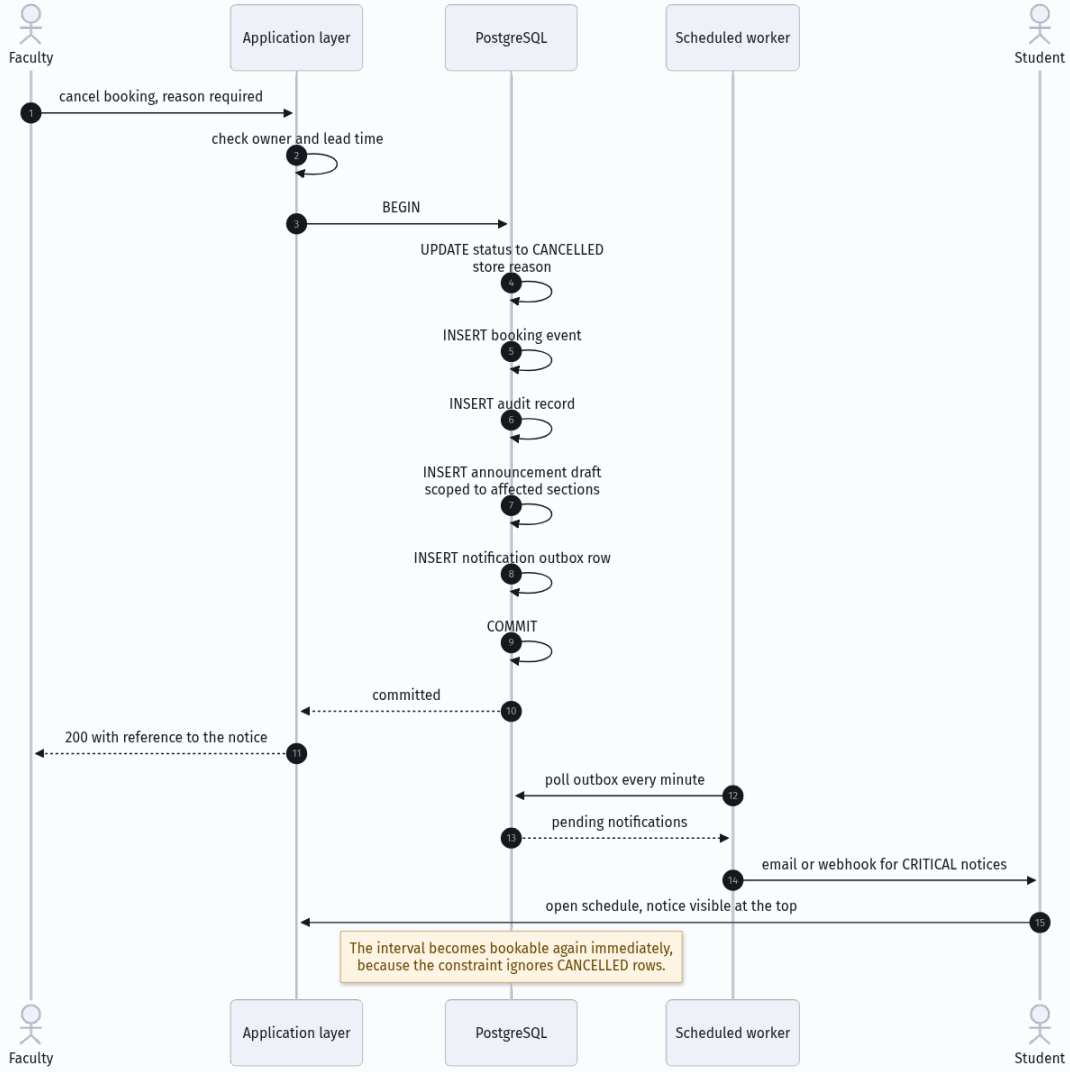


FIGURE 13 *Cancellation. Status, history, audit, notice and outbox row commit together; delivery happens afterwards, so a mail failure cannot undo the cancellation.*

7.6 Natural-language search, grounded

The language model translates an utterance into a structured query and explains what came back. It is placed behind the same engine as the form, and its output is checked against the rows that were actually returned before the answer is displayed.

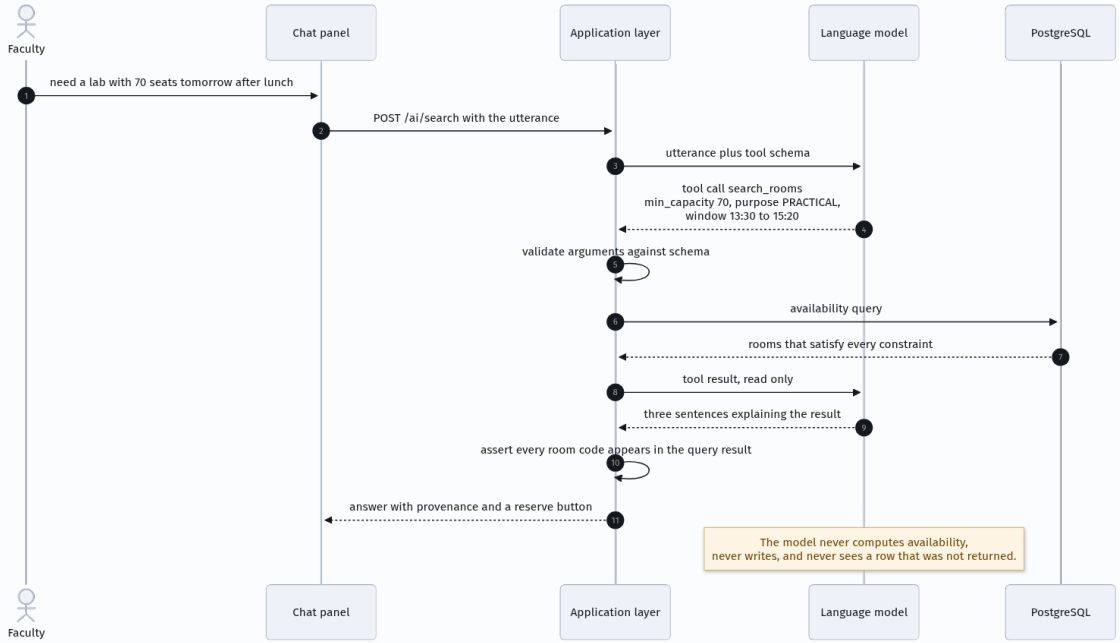


FIGURE 14 *Grounded search. The model sees only the result of a query this system ran, and it has no tool that writes.*

The guardrail at the end of that sequence is worth naming. Before the answer reaches the interface, the server checks that every room code the model mentions appears in the result set it was given. If it mentions a room that was not offered, the answer is replaced by the plain result list. That check costs a few lines and makes the hallucinated-room failure impossible rather than unlikely.

PART III

Evidence and behaviour

How this compares with current practice, what the arithmetic says, and what happens at the boundaries.

— 8 Comparison with the methods in use today
— 9 Comparative figures
— 10 Edge cases
— 11 Quality: non-functional requirements and testing

SECTION 8

Comparison with the methods in use today

It would be easy, and unfair, to describe current practice as chaotic. It is not. It is a set of reasonable local solutions to a problem nobody has centralised: a group chat for the department, a notice board outside the office, a spreadsheet that one person maintains, and the corridor conversation that settles most cases in under a minute. Each of these works for the single case in front of it. What they cannot do is answer a question that spans all of them.

8.1 Four methods, one criterion each

Criterion	Word of mouth and corridor enquiry	Notice board	Shared spreadsheet	This system
Answers a question about tomorrow	Sometimes, if the right person is present	No, the board shows notices, not availability	Yes, if recently updated	Yes, computed at the moment of asking
Prevents two claims on one room	No	No	Only within the sheet, and only if one person edits at a time	Yes, in the database
Prevents a section being sent to two rooms	No	No	No	Yes, by constraint
Reaches the students affected by a change	Relayed second-hand	Only readers of the board	No notification	Notice generated from the change itself
Survives a disagreement afterwards	Memory	Paper, if kept	Cell history, if saved	Full audit trail with before and after values
Works as well at 8:55 as at 11:00	No	No	Yes	Yes
Cost to a new member of faculty	Must know whom to ask	Must walk to the board	Must ask for edit access	Nothing, if they have an email address
Fails quietly or loudly	Quietly	Quietly	Quietly	Loudly, and with an alternative offered

TABLE 11 *Current practice compared on eight criteria. The decisive row is the last: systems that fail quietly produce the incident this project exists to prevent.*

The spreadsheet deserves a fairer hearing than it usually gets, because it is the only current method that could in principle do the job. What defeats it is concurrency. Two people with the file open produce two versions, and the later save wins silently. The same failure, stated precisely, is why this design puts the conflict rule in the database: simultaneous access is not a rare condition here, it is the normal condition on the morning a room is scarce.

8.2 The cost of coordinating without a register

When n people need to agree about a shared resource and none of them can consult a record, every pair of them is a possible conversation. The number of distinct channels is n(n-1)/2, which grows with the square of the group. A register collapses this to n: each person consults one record, and the

record holds the agreement.

Faculty coordinating room use	Possible pairwise conversations	Queries against one register
10	45	10
20	190	20
30	435	30
40	780	40

TABLE 12 *Coordination paths. At thirty members of faculty the pairwise figure is fourteen times the centralised one, and most of those conversations happen because the answer was not written down.*

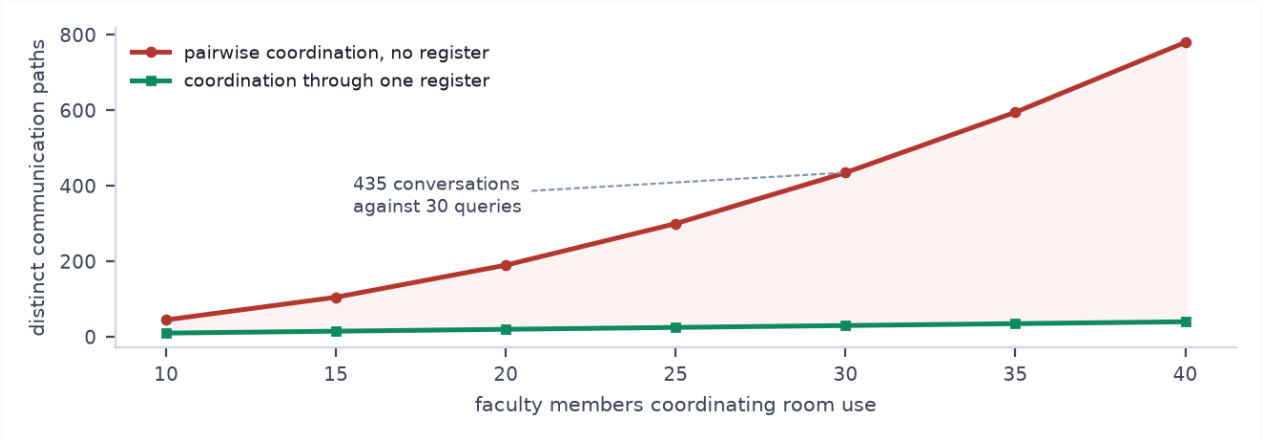


FIGURE 15 *The same comparison as a curve. The gap between the two lines is the coordination work a register removes, not by working faster but by removing the work.*

8.3 How often uncoordinated booking produces a clash

This is the part of the argument that surprises people. Clashes do not require bad luck; they require only that bookings are made without reference to each other. The arithmetic is the same as the birthday problem. If N bookings are made independently across M possible room-terms, the expected number of colliding pairs is

$$E = N\,(N-1)\,/\,2M$$

and, for a Poisson approximation, the probability that at least one clash occurs on a given day is

$$P(\text{at least one clash}) \approx 1 - \exp(-N\,(N-1)\,/\,2M)$$

With thirty bookable rooms and eight periods, M is two hundred and forty. At thirty uncoordinated booking attempts a day, the expected number of colliding pairs is 1.8 and the probability of at least one clash on a given day is eighty-four per cent. Over an eighteen-week semester of six-day weeks that is close to two hundred clashes, before anyone has made a mistake.

Booking attempts per day	Expected colliding pairs per day	Probability of at least one clash	Expected clashes per semester
10	0.19	17%	20
20	0.79	55%	85
30	1.81	84%	196
40	3.25	96%	351

Booking attempts per day	Expected colliding pairs per day	Probability of at least one clash	Expected clashes per semester
60	7.38	99.9%	797

TABLE 13 *Clash likelihood as a function of volume. With the exclusion constraint in place, the right-hand column is zero by construction rather than by effort, and that is the difference the design buys.*

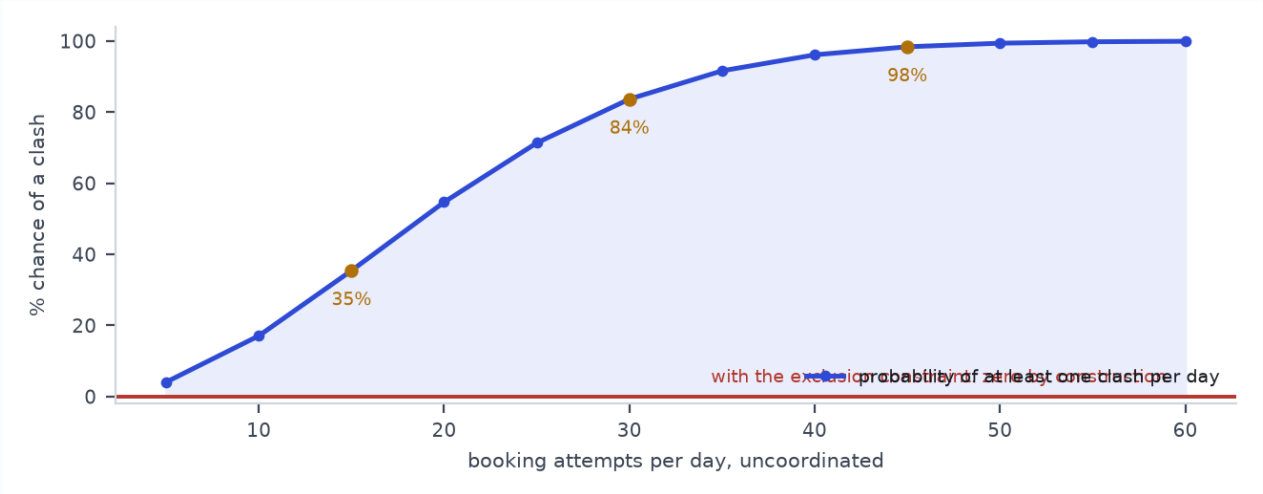


FIGURE 16 *The same table as a curve, with the three volumes marked. The shape matters more than any single point: the risk rises steeply and then saturates.*

Two caveats, because a model presented without them is not a model. The estimate assumes attempts are independent and roughly uniformly spread across room-terms; in reality demand clusters into popular rooms and popular periods, which makes clashes more likely than the figures suggest, not less. It also counts only those conflicts that occur within the record; the failures that hurt students are the ones where the record itself is wrong or missing, and no probability estimate covers those.

8.4 Time to find a room

Step	Today, by enquiry	Today, by spreadsheet	This system
Establish who knows	2 to 5 minutes, or a walk	Depends on access	None
Establish what is free	5 to 10 minutes, often by walking past rooms	2 to 5 minutes, if the file is current	One query, under 100 ms
Check the room suits the class	From memory	From a comment, if present	Capacity, features, floor and walking time shown together
Secure it	Verbal agreement, unrecorded	A cell edit, unrecorded	A reservation, audited
Tell the people affected	Separate action, easily forgotten	Separate action, usually forgotten	Automatic, inside the same transaction
Typical total	15 to 30 minutes, in sequence	5 to 15 minutes	Under a minute, and the notice is already sent

TABLE 14 *Finding a room, step by step. The saving is less in the query than in the last two rows, which are the rows that currently get skipped.*

8.5 What changes, in one table

Property	Current practice	This system	How it is evidenced
Double booking	Occurs; probability rises with volume	Impossible for live reservations	Twenty parallel identical requests: one success, nineteen refusals
Section clash	Undetected until students arrive	Prevented by constraint	Constraint test on the section key
Change notification	Manual, and therefore occasional	Generated by the change	End-to-end test: cancel, then read the section's schedule
History	Memory, or a saved file	Append-only audit with before and after values	Replay the log and reconstruct any room's state
Alternatives	Ask again tomorrow	Three ranked, revalidated options per refusal	Property test over five hundred generated booking sets
Room data accuracy	Varies by person	Single registry, ambiguities flagged	Five documented reconciliations, one click to confirm

TABLE 15 *Summary of the change, with the test that demonstrates each line.*

KEY Current practice fails quietly. That single property, more than any feature list, is what makes a central register worth building.

SECTION 9

Comparative figures

The figures in this section are derived from the supplied room register and the branch strengths in the problem statement. Where an input is an assumption, it is named. The purpose is not to decorate the document with numbers but to show which design decisions the numbers force, and to give a future maintainer a way to check whether the system still matches the campus.

9.1 Supply and demand

Quantity	Value	Derivation
Bookable rooms	30	Sixteen in IET, fourteen in Deeksha Bhawan
Teaching periods per day	8	09:00 to 17:10, with lunch from 12:40 to 13:30
Room-period supply per day	240	30 rooms × 8 periods
Sections	32	Six branches over four years: 2+2+1+1+1+1 per year
Students	2,400	600 per year × 4
Largest teaching unit	75	Every section is seventy-five strong
Rooms that seat 75 or more	20	Fourteen IET rooms, the seminar hall, the library, and four long rooms in Deeksha Bhawan
Rooms that seat a spaced 75-student examination	2	Seminar hall at 100 spaced seats; library at 90

TABLE 16 *The inputs. Everything in this section follows from this table.*

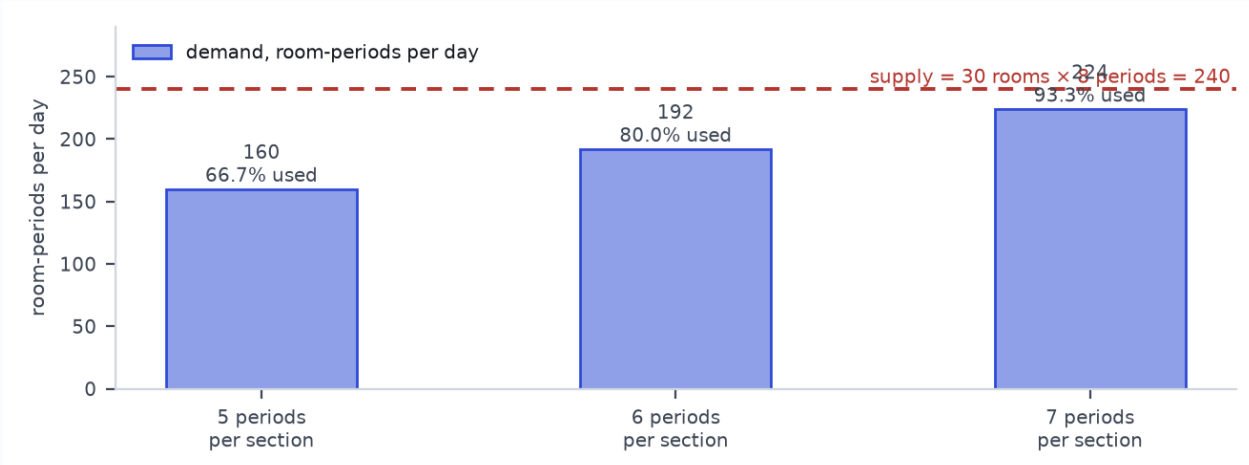


FIGURE 17 *Demand against supply at three timetable loads. At seven periods a day the campus runs at ninety-three per cent of its theoretical maximum, which leaves no room for an examination, an event or a repair.*

9.2 The peak-hour problem

Aggregate utilisation understates the difficulty, because demand is not spread evenly across the day. Morning periods and the period after lunch carry most of the load. If fifty-five per cent of a day's activity wants three of the eight periods, those three periods see about thirty-five concurrent demands against thirty rooms.

Load	Demand per day	Concentration	Concurrent demand in peak slots	Against 30 rooms
5 periods per section	160	50% across 4 slots	20.0	0.67, comfortable
6 periods per section	192	50% across 4 slots	24.0	0.80, workable
6 periods per section	192	60% across 4 slots	28.8	0.96, fragile
6 periods per section	192	70% across 4 slots	33.6	1.12, oversubscribed
6 periods per section	192	55% across 3 slots	35.2	1.17, oversubscribed
7 periods per section	224	70% across 3 slots	52.3	1.74, badly oversubscribed

TABLE 17 *Peak pressure under different assumptions. Above about sixty per cent concentration the campus is short of rooms in the slots everybody wants, and no amount of interface polish changes that.*

This is the figure that justifies the alternative-suggestion engine. When demand exceeds supply in a slot, the useful answer is not "no rooms available"; it is a ranked list of the least bad options, each with its cost stated. That is also why the engine is allowed to shift a window by up to half an hour: the third-best room at the requested time is frequently worse than the best room twenty minutes later.

9.3 The examination case

Alternate-seat spacing halves usable capacity. Applied to this campus, the result is blunt.

Room	Nominal capacity	Spaced capacity at 0.5	Seats a full section of 75?
Seminar hall IET32	200	100	Yes
Library	180	90	Yes
LH01, LH02, LH08, LH09	80	40	No, on its own
Fourteen IET rooms	75	37	No
LH06, LH07, LH13, LH14	55	27	No
LH03, LH04, LH05, LH10, LH11, LH12	45	22	No

TABLE 18 *Examination capacity by room class. Two rooms out of thirty can seat a section for an examination without splitting it.*

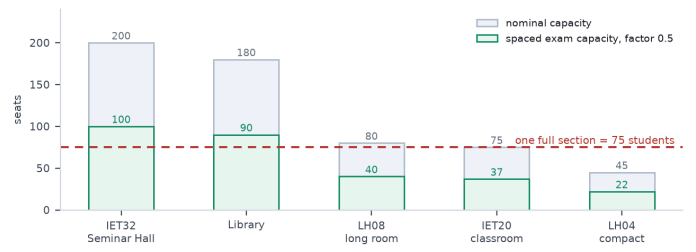


FIGURE 18 *Nominal against spaced capacity for representative rooms. The line at seventy-five is one section, and only two bars clear it.*

The planning consequence is worth stating in sittings. For one subject taken by all thirty-two sections, the two qualifying rooms give sixteen sittings, which at three examination periods a day is six days of examinations for a single subject. If the system is allowed to split a section across two rooms, the arithmetic improves considerably, with one important catch.

Arrangement	Simultaneous sections	Sittings for 32 sections	Days at three sittings a day
No splitting: seminar hall and library only	2	16	6
Splitting using the four long rooms in Deeksha Bhawan as two pairs of 40 plus 40	4	8	3
Splitting two 37-seat classrooms together	not possible	—	two classrooms hold 74 between them, one short of a section

TABLE 19 *What splitting actually buys. Note the third row: two ordinary classrooms cannot host one section between them, so a split needs either the long rooms or a third room.*

That last detail is the kind of thing that is only visible once the numbers are written down. It also explains the incident in the problem statement, and it turns a vague requirement about intelligent assistance into something specific: the system must be able to plan a split sitting, name the two rooms, notify both sections, and record the arrangement as one decision rather than two unrelated bookings.

9.4 What a cancellation actually recovers

The original proposal expected cancellations and low attendance to release rooms. The recoverable time depends almost entirely on when the release happens.

Release happens	Remaining time in the period	Usable by another class?	Recovered
At the start of the period	55 minutes	No: the class is running	0
10 minutes in	45 minutes	No	0
20 minutes in	35 minutes	No	0
30 minutes in	25 minutes	No: too short for a period	0
The day before	55 minutes	Yes, a whole period	100% of the slot
Two days before, with a notice	55 minutes	Yes, and the affected section can be told in time	100% of the slot

TABLE 20 *Recovery by timing. Only planned release recovers anything, which is why the system supports planned cancellation and notices rather than a release button pressed during a lesson.*

9.5 Utilisation, and what sits idle

Timetable load	Room-periods used per day	Utilisation	Idle room-hours per day
5 periods per section	160	66.7%	73.3 hours
6 periods per section	192	80.0%	44.0 hours
7 periods per section	224	93.3%	14.7 hours

TABLE 21 *Idle capacity, assuming a fifty-five minute period. The middle row is the important one: forty-four room-hours a day sit unused while faculty report that no room is available, which is a distribution problem rather than a shortage.*

9.6 Assumptions, and how to check them

Every figure above rests on the register and on four assumptions. We would rather state them than have them discovered.

- A period is fifty-five minutes and there are eight of them. If the institute runs a different grid, the supply figure changes and every percentage with it; the grid is a setting, not a constant.
- Each section is taught as one block of seventy-five. If a section is routinely split for practicals, the effective demand per room drops and the peak pressure eases; the scoring function already prefers the tightest fit, so the effect appears in the recommendations without any code change.
- Spacing halves capacity. If the institute's examination norms allow closer seating, the number of qualifying rooms rises and the six-day figure falls; the factor is one number in settings.
- Bookings are spread across room-terms. As noted in section 8.3, real demand clusters, so the clash figures are a floor rather than a ceiling.

The health endpoint reports the register size, the section count and the current settings values, so a maintainer can re-run this section's arithmetic against live data in a couple of minutes rather than trusting a document written months earlier.

SECTION 10

Edge cases

A design is judged less by its happy path than by what it does at the boundaries. The cases below are the ones we expect to meet, most of them in the first month of use. For each, the behaviour is decided in advance, which is the only way to avoid deciding it at nine in the morning with a class waiting.

Case	What the system does	Why
One reservation ends at 11:00 and another begins at 11:00	Both stand. They do not conflict	Intervals are half-open, so the end instant belongs to the next interval. Back-to-back classes are the commonest pattern in any timetable
Two faculty submit the same room and window in the same second	The database refuses the second insert; the caller receives three alternatives	The exclusion constraint, not the application, is the arbiter
The same person double-clicks the confirm button	The request carries an idempotency key, so the second call returns the original reservation instead of creating a duplicate	Double submission is indistinguishable from intent at the network level, so it is solved with a key rather than with a disabled button
A section is already booked in another room at that hour	Refused, and the existing reservation is named in the refusal	A section in two rooms is invisible to a room-only check and is what students experience as an unexplained move
The member of faculty is already booked elsewhere at that hour	Refused, with the conflicting reservation named	Usually discovered by a class with no teacher in it
A request for exactly seventy-five students in a seventy-five-seat room	Accepted, with no slack and no warning	Exact fits are legitimate; the system warns only when a request exceeds capacity. Adding a guest later is a new decision, not an error in the old one
An examination request that fits the room but exceeds its spaced capacity	Refused for the examination purpose, with a split sitting proposed	A seventy-five-seat room holds thirty-seven candidates with spacing; treating nominal capacity as examination capacity is how candidates end up without a desk
Cancellation is attempted after the start time	Refused for the requester; an administrator may still record it, with a reason	The class has happened or is happening. Rewriting it serves nobody except in a correction, which is an administrative act
A class finishes early and the room is offered for the rest of the period	Allowed as an early release by the owner only, and the remainder of the interval returns to the pool	Partial release is honest about what is available, unlike releasing the whole period
A room is marked closed for repair while reservations exist inside the closure	The closure cannot be saved until each affected reservation is cancelled or moved, and the affected list is shown first	Silently breaking three bookings to record one repair is the wrong priority

Case	What the system does	Why
A timetable change moves a class into a room that is already reserved	The change is queued for an administrator with both reservations shown	The official timetable and the ad-hoc reservation have equal standing; a person decides between them
An approval request is neither approved nor rejected	It expires after forty-eight hours, and the requester and the approver are told	Pending requests hold the interval. An abandoned request must not block a room for a week
Two administrators approve the same request	The first commit wins; the second receives a clear refusal, not a duplicate	The constraint protection extends to status changes
An administrator must take a room for an examination	A displacement is recorded with a reason, the notice is generated, and the displaced requester is told where to go instead	Overrides are legitimate and must leave a trace; a silent override is the failure mode this project exists to remove
A published notice is corrected	A new version is created, the original remains readable, and the notice shows when it was last updated	Students who read yesterday's version need to see that it changed
Twelve students report the same wrong room	One triage item is created with a count of twelve	Twelve separate items waste the administrator's attention; a count is the signal
A report concerns a reservation that has since been cancelled	Closed automatically as no longer applicable, and the reporter is told	Leaving stale reports open trains reporters to stop filing them
A requested room does not exist	Refused at validation, before any query	This is the failure a language model can introduce and a schema check cannot
The language model returns a room that was not in its result set	The answer is discarded and the plain result list is shown	The guardrail is a set comparison, not a matter of trusting the model
A user's browser is set to a different time zone	All campus times are displayed in the campus zone and labelled as such	A booking at 10:00 is at 10:00 in Gorakhpur regardless of where the browser thinks it is
An event runs past midnight	Recorded as one interval across the boundary	Splitting a reservation at midnight introduces a class of bug that no user asked for
An unconfirmed room is later confirmed with a smaller capacity	Future reservations in that room are re-checked, and any that no longer fit are listed for the administrator	Correcting reference data must not silently create a capacity breach
A member of faculty leaves, and their future reservations remain	Reservations are retained and the owner is shown as former staff; an administrator reassigns or cancels	Deleting a person must not delete the record of what they promised
A series of weekly classes contains one conflicting instance	The series is created with that instance excluded and listed, rather than failing as a whole or skipping it silently	All-or-nothing is unhelpful; silent skipping is worse
A student has no signal at the door	The day's schedule is printable, and the page is usable after a single load	The system should not make a corridor's connectivity a prerequisite for attending a class

TABLE 22 *Twenty-six cases, with the behaviour and the reason for it. Each one is a test in the suite described in section 11.*

10.1 Three cases that deserve more than a table row

Simultaneous access is the normal case, not the exception

Most booking software treats two people acting at once as a rare condition and checks for conflicts in application code, which passes both requests and then writes both rows. On this campus the condition is ordinary: the period before lunch on the day an examination schedule is published is precisely when several people want the same two rooms. The design treats it as the base case. The check is a constraint that the storage engine enforces inside the insert, so there is no window between checking and writing in which another request can slip through. We verify it with twenty identical parallel requests and assert that exactly one succeeds.

Partial availability is a real answer

A faculty member asking for ten to eleven on a Thursday can be told three different things: yes, no, or yes from half past ten. The third answer is the most useful and the hardest to compute, because it requires looking for gaps rather than for rooms. The system returns the largest free interval inside the requested window and, when that interval is shorter than the request, proposes either a shortened activity or a different room. This is also why the reservation model stores arbitrary intervals rather than snapping everything to the eight-period grid: the grid exists for the official timetable, and the rest of the campus does not live on it.

A split sitting is one decision, not two bookings

Splitting a section across two rooms for an examination is not two unrelated reservations that happen to share a time. If one of the two rooms is later closed, the whole sitting is affected, and the notice must reach all seventy-five students. The model therefore groups the parts: one parent record carries the purpose, the subject and the section, and the parts carry the rooms. Cancellation, notification and audit act on the group. Without that grouping, the half of the section in the closed room receives a notice and the other half does not, which is a new way to reproduce the original incident.

SECTION 11

Quality: non-functional requirements and testing

11.1 Non-functional requirements

Area	Requirement	How it is met
Accessibility	The student view is usable on a mid-range phone, in daylight, in a corridor	Server-rendered pages, no client-side data fetching for the schedule, a single content column, and a printable day view
Contrast and keyboard use	Contrast ratio of at least 4.5 to 1, and every action reachable without a mouse	Checked at the end of each interface change rather than at the end of the project
Time handling	One campus time zone, correct across browsers and devices	All boundaries stored with time zone; all displays labelled Asia/Kolkata
Startup	The application starts with only a database connection string	Everything else, including the language model and the mail provider, is optional and degrades
Availability	A failed notification never undoes a reservation	Outbox pattern: the notification row commits with the reservation and is delivered afterwards
Security	Sessions in http-only cookies, passwords hashed with a modern key-derivation function, rate limits on reports and on the language-model endpoint	No secret in the repository; environment variables validated at startup
Auditability	Every mutation is attributable and replayable	Append-only audit table, written in the same transaction, with before and after values
Recoverability	The demonstration state can be rebuilt from the repository alone	Idempotent seed from a versioned registry file, plus a single command that removes test records

TABLE 23 *Non-functional requirements, stated as properties rather than as intentions.*

11.2 Testing

Layer	What it covers	The failure it catches
Unit	Interval arithmetic, adjacency against overlap, capacity boundaries, every term of the scoring function, period conversions	The two classic defects: back-to-back bookings treated as a clash, and an eighty-seat room offered to eighty-one students
Integration, against a real PostgreSQL instance	Two concurrent inserts, cancellation freeing an interval, closures blocking a slot, approval-gated purposes landing pending	The race condition, and the cancelled-but-still-blocked bug
Policy	The full permission matrix, every cell including the refusals	Privilege leaks, which are the most damaging kind of defect in a demonstration

Layer	What it covers	The failure it catches
End to end	Four journeys: book a room; meet a conflict and take an alternative; cancel and observe the notice; attempt to book as a student and be refused	Breakage between layers, which unit tests cannot see
Property-based	Five hundred generated booking sets, asserting that no recommended room overlaps an existing reservation	Confidently wrong recommendations
Load	Two hundred virtual users for sixty seconds, with seventy per cent of requests aimed at three popular rooms	Both the latency budget and the concurrency claim, with numbers to show

TABLE 24 *The test layers, and the specific defect each one is there to catch.*

11.3 Acceptance criteria, mapped to the problem statement

Requirement in the statement	Acceptance criterion we will demonstrate
Room availability across both buildings	A fully booked IET first floor returns no rooms; widening to both buildings returns Deeksha Bhawan rooms with distance shown
Faculty booking for four purposes	A class, an examination, a practical and an event each persist with the correct purpose and policy
Conflict prevention with alternatives	Twenty parallel identical requests produce one success and nineteen refusals, each carrying three usable alternatives
Schedule updates and announcements	Cancelling a reservation updates the affected section's schedule and adds a notice within one refresh
Student access without modification	A student session can read, comment and report; a direct modification attempt is refused
Role-based access	The full matrix is asserted, including the negative cells
Accountability and history	Replaying the audit log reconstructs any room's state at any past time
Intelligent assistance, validated	Every recommendation is re-checked against the reservation table before it is displayed

TABLE 25 *Each requirement from the problem statement, with the demonstration that settles it.*

PART IV

Delivery and reference

Build pipeline, risk, the questions we expect, and the reference material a maintainer needs.

— 12 Delivery: repository, pipeline and deployment
— 13 Risks and mitigations
— 14 Frequently asked questions
— 15 Appendices: room register, diagram sources, derivations, glossary

SECTION 12

Delivery: repository, pipeline and deployment

The event's title puts DevOps beside artificial intelligence, and the deliverable asks for a repository and setup instructions. That part is treated as a first-class requirement rather than as packaging.

12.1 Repository layout

```
smart-campus/
app/ route handlers and pages, one deployment
api/rooms/availability/ the query in Listing 1
api/bookings/ create, reschedule, cancel
api/bookings/[id]/alternatives/
api/announcements/ api/reports/ api/ai/search/
lib/
db/ schema, client, migrations
policy/can.ts one authorisation function
scheduling/ availability, conflicts, scoring, allocation
time/intervals.ts pure interval arithmetic, no dependencies
ai/ adapter, tool schema, prompts
validation/ schemas shared by forms, routes and tools
data/
campus_room_registry.json the room register, versioned
seed/ students, timetable, demonstration bookings
scripts/ ingest, seed, reset demonstration data
tests/ unit, integration, end to end, load
docs/ this document, decisions, setup
.github/workflows/ci.yml
docker-compose.yml local PostgreSQL
README.md SETUP.md ARCHITECTURE.md
```

CODE *Listing 6. Repository layout. Two files carry disproportionate weight: the policy function, because every route depends on it, and the interval module, because that is where the subtle defects live.*

12.2 The pipeline

Stage	What runs	Gate
1 Verify	Lint and type check	Any error stops the pipeline
2 Unit	Interval, scoring and policy tests	Any failure stops the pipeline
3 Migrate and seed	Migrations applied to a temporary database, including the three exclusion constraints; register and students loaded	A migration that cannot run against an empty database is a defect
4 Integration	Concurrent inserts, closures, approval transitions	As above
5 End to end	The four journeys, in a browser	As above
6 Preview	A deployment with its own URL for the branch	Automatic; the URL is quoted in the pull request
7 Merge	Reviewed and merged to main	Review plus all gates green
8 Production	Deployment from main	Automatic on merge

Stage	What runs	Gate
9 Load	Two hundred virtual users, conflict count must be zero	Runs on main and on demand, not on every branch

TABLE 26 *Nine stages. The first five are the ones that catch defects; the rest exist so that a defect found in one place cannot reach a person.*

12.3 Configuration that must not be hard-coded

Six values decide behaviour and all six are settings, because all six are things an institute may change without changing the software: the period grid and the lunch break, the examination spacing factor, the minimum gap for a cross-building move, the booking horizon and lead time, the approval thresholds, and the scoring weights. Each is stored with its current value and its previous values, and changing one writes an audit record.

12.4 Definition of done

- A deployed address that a reviewer can open, with three demonstration accounts printed on the sign-in page.
- A green pipeline on the main branch, with the load test output attached to the release.
- Double booking shown to be impossible by the concurrency test rather than asserted in prose.
- All eight requirements from the problem statement demonstrable in a single seven-minute walkthrough.
- Every seeded record carrying a visible test marker, and one command that removes them.
- A README that takes a stranger from a clone to a running application in under five minutes, on a machine that has only Node and Docker.
- The five source ambiguities documented in this document and visible in the interface as flags rather than as silent assumptions.

SECTION 13

Risks and mitigations

Risk	Likelihood	Mitigation	If it happens anyway
The student dataset's columns differ from expectation	High	Six-stage ingest with branch alias mapping, roll-number detection and a dry-run diff that reports every rejected row	Ship the register plus a generated student set, labelled as generated, and say so
The five blueprint ambiguities are challenged	High	Reconciliation ledger in section 6.5, a flag on each affected room, and one administrative screen to confirm them	Answer with the ledger; a documented resolution is stronger than a hidden guess
The object-relational mapper cannot express the exclusion constraint	Medium	Hand-written SQL migration, written and tested on the first day	Serialisable transactions with retry, which is correct and slightly slower; publish the trade-off
The language-model provider is unavailable during the demonstration	Medium	Provider-agnostic adapter, eight-second timeout, and the form as the primary path	Nothing to apologise for: the form is the primary path by design
Integration failure on the final evening	Medium	Interface contract fixed on day one, interface built against fixtures from day two, features frozen two days before the demonstration	Demonstrate the tagged release from the previous day; an older working build beats a newer broken one
Data loss before the demonstration	Low	Database dump before the demonstration, idempotent seeds in the repository, rehearsed restore	Restore in half a minute; the rehearsal is what makes that possible
Scope creep, particularly into attendance or results	Medium	The out-of-scope list in section 1 is treated as binding; additions need a corresponding removal	Refuse the addition and record why; a smaller complete system scores better

TABLE 27 *Risks, ordered by likelihood rather than by severity, because the likely ones are the ones that actually happen.*

SECTION 14

Frequently asked questions

These are the questions we have been asked, or expect to be asked, in roughly the order they arrive.

Why not simply store whether a room is free or occupied?

Because a room's state is not a value, it is a schedule. A flag can answer whether somebody is in the room now, and that is the one question nobody needs to ask. Every useful question is about a window: is this room free tomorrow between ten and eleven, and for how much of it. Once the answer has to include a window, the flag becomes an interval, and the interval is what the database stores.

Why PostgreSQL rather than MySQL or MongoDB?

Two reasons, both practical. PostgreSQL can enforce the overlap rule in the storage engine with an exclusion constraint over a range type, which is the difference between a conflict being impossible and a conflict being unlikely. It also has the range types, indexes and time-zone handling that this problem needs without extensions beyond one. MySQL cannot express that constraint; MongoDB would move the check back into application code, which is where the race lives.

Attendance has been removed from the model. What happens when a class is cancelled?

The member of faculty cancels the reservation, with a reason. That releases the interval, generates a notice to the affected sections, and writes an audit record. It does not touch attendance, which remains a departmental record of who was present. Keeping the two apart is what stops a cancelled class and an empty class from looking identical, which is the ambiguity that caused the original incident.

Where do the capacities of the Deeksha Bhawan rooms come from?

Nowhere, in the supplied material. Neither the statement nor the plan gives them. We estimated from relative floor area: the long rooms at eighty, the medium rooms at fifty-five, the compact rooms at forty-five. They are stored as configurable values and marked as assumed in the interface until an administrator confirms them, which is a single screen.

What genuinely happens if two people book at the same moment?

One succeeds and the other receives a refusal with three alternatives. The refusal comes from the database, not from application logic, so there is no interval between checking and writing in which a second request can slip through. We demonstrate this with twenty identical parallel requests and count the outcomes.

Can a room be released in the middle of a period?

It can be released for the remainder of the period, by the person who holds it, and that remainder genuinely returns to the pool. What the system will not do is treat a room as free for a window that overlaps the class still sitting in it. The distinction sounds pedantic until you watch a class arrive at a room that a screen told them was empty.

Why is the system allowed to move the time of a booking when it suggests alternatives?

Because the honest answer to a clashing request is often not another room. Our scoring shows a better room twenty minutes later beating the best room at the requested time, and in a campus where peak periods are oversubscribed by half again, that is usually true. The shift is bounded at half an hour and always stated explicitly, so nobody is moved without being told.

Does splitting an examination across two rooms not double the invigilation?

It does, and on this campus it is sometimes the only way to seat a section at all. Two hundred candidates aside, a seventy-five-seat room holds thirty-seven with spacing, so a section of seventy-five cannot sit in an ordinary classroom. The system's job is to make that visible early, propose the split with named rooms, and notify all the students at once, rather than to let it be discovered on the morning.

What if the language model suggests a room that does not exist?

It cannot reach the interface. The model's only output that matters is a structured query, validated against a schema before it is executed, and its explanatory sentence is checked against the result set before display: any room code that was not in the result causes the answer to be replaced by the plain list. The model has no tool that writes.

What happens if the internet fails during a class change?

The schedule page is server-rendered and usable after one load, and the day view prints. The system does not make attendance at a class conditional on a corridor's signal. Reservations made while offline are not accepted, which is the correct limitation for a system whose value depends on a shared record.

How do students find out that something changed?

The change generates the notice. Cancelling or moving a reservation writes a notice scoped to the sections affected and queues a notification, inside the same transaction. Critical notices appear at the top of the student feed and are sent by mail or webhook. A change cannot be made silently, because the notice is not a separate step somebody may forget.

Why should students not be allowed to book rooms?

Because the resource is scarce and the allocation follows the timetable. Students are given a voice without authority: they can comment, and they can file a structured report, which goes to a queue with duplicates merged and a resolution notified back to them. That is enough to correct the record quickly without turning rooms into a first-come scramble.

How are the five ambiguous facts about the buildings handled?

They are recorded in section 6.5 with the reasoning, they are flagged on the room rows, they appear in the interface as a marker, and an administrator can confirm each one in a single click. We chose to state the uncertainty rather than to resolve it quietly, because each one is a decision somebody can check.

Why not build a mobile application?

Because the student experience is a web page that loads in a second, and a native application would consume a week and reach the same people. The interface is designed mobile-first; the packaging is not where the value is.

What stops a member of faculty from reserving rooms weeks ahead and hoarding them?

Three things: a booking horizon of sixty days, an approval gate on the large venues, and a fairness term in the scoring that prefers rooms which have been used less during the week. Fairness is a soft weight rather than a hard rule, deliberately, because a genuine examination must still be able to claim the seminar hall.

What happens when a room goes out of service for repairs?

A closure is recorded against the room and the interval. The closure cannot be saved while reservations exist inside it: the affected reservations are listed first, and each must be cancelled or moved. Breaking three bookings quietly to record one repair would be the wrong priority.

Is this only for the Institute of Engineering and Technology?

The model already carries both buildings, and the register is a data file rather than a schema full of special cases. Adding a third building means adding rooms and a walking distance, not changing

code. That was a deliberate constraint on the design: anything specific to IET lives in data.

How do you demonstrate that a student cannot modify a booking?

By replaying a faculty request with a student's own session and asserting the refusal, in the test suite. Hiding a button in the interface is a convenience; the refusal at the route is the security, and it is the thing we show.

What if the official timetable changes in the middle of a semester?

Timetable entries carry a validity window, so a change applies from a date rather than retroactively. Where a new timetable entry collides with an existing reservation, the collision is presented to an administrator with both records shown; neither the timetable nor the reservation wins automatically, because both represent a real promise.

Why not microservices, or an event-driven architecture with a message broker?

Because the correctness of this system depends on one transaction doing several things at once: inserting the reservation, checking the constraint, writing the history and queuing the notice. Splitting that across services would introduce the distributed consistency problem in exchange for scale this campus does not need. The one place asynchrony earns its keep, notification delivery, is handled with an outbox table and a single scheduled job.

What has been deliberately left out?

Attendance and results, which belong elsewhere; room-steward rotas and inventory; fee and transport systems; a native application; single sign-on, which would consume a day and add nothing to the demonstration; and any machine-learning model trained locally, because a deterministic scoring function with visible weights is easier to defend and easier to fix.

SECTION 15

Appendices

Appendix A. The room register

Thirty-four records, of which thirty are bookable. Capacities in the examination column apply the spacing factor of one half. Where a value was neither stated in the problem statement nor readable from the plan, it is marked assumed and carries a flag in the data so that it can be confirmed.

Code	Building	Floor	Type	Capacity	Exam	Bookable	Flag
LIB	IET	GF	Library	180	90	Yes	
TO	IET	GF	Office	—	—	No	
DO	IET	GF	Office	—	—	No	
PC	IET	GF	Office	—	—	No	
IET22	IET	FF	Lab	75	37	Yes	Unconfirmed
IET16	IET	FF	Lab	75	37	Yes	Unconfirmed
IET14	IET	FF	Lab	75	37	Yes	Unconfirmed
IET15	IET	FF	Lab	75	37	Yes	Unconfirmed
IET20	IET	FF	Class	75	37	Yes	
IET19	IET	FF	Class	75	37	Yes	
IET18	IET	FF	Class	75	37	Yes	
IET17	IET	FF	Class	75	37	Yes	
IET32	IET	SF	Seminar	200	100	Yes	
IET24	IET	SF	Class	75	37	Yes	
IET25	IET	SF	Class	75	37	Yes	
IET30	IET	SF	Class	75	37	Yes	
IET29	IET	SF	Class	75	37	Yes	
IET28	IET	SF	Class	75	37	Yes	
IET23	IET	SF	Class	75	37	Yes	Unconfirmed
IET-SF-STORE	IET	SF	Service	—	—	No	Unconfirmed
LH01	DB	GF	Class	80	40	Yes	
LH02	DB	GF	Class	80	40	Yes	
LH03	DB	GF	Class	45	22	Yes	
LH04	DB	GF	Class	45	22	Yes	
LH05	DB	GF	Class	45	22	Yes	

Code	Building	Floor	Type	Capacity	Exam	Bookable	Flag
LH06	DB	GF	Class	55	27	Yes	
LH07	DB	GF	Class	55	27	Yes	
LH08	DB	FF	Class	80	40	Yes	
LH09	DB	FF	Class	80	40	Yes	
LH10	DB	FF	Class	45	22	Yes	
LH11	DB	FF	Class	45	22	Yes	
LH12	DB	FF	Class	45	22	Yes	
LH13	DB	FF	Class	55	27	Yes	Unconfirmed
LH14	DB	FF	Class	55	27	Yes	Unconfirmed

TABLE 28 *The register as seeded. Four records are not bookable: three offices and one unlabelled service room on the second floor.*

Appendix B. Diagram sources

Each figure in this document was produced from a text source, so that it can be corrected without redrawing anything. The sources for the two diagrams that carry the core argument are reproduced here; the remainder live in the repository alongside this document.

```
flowchart TD
Q["Is IET20 free tomorrow 10:00 to 10:55?"] --> D{"derived query"}
D --> R1["any CONFIRMED booking overlapping the window?"]
D --> R2["any TimetableSlot overlapping the window?"]
D --> R3["any RoomClosure overlapping the window?"]
R1 --> U["UNION of occupied intervals"]
R2 --> U
R3 --> U
U --> A{"window INTERSECTS occupied?"}
A -->|"yes"| NO["not available: return three ranked alternatives"]
A -->|"no"| YES["available: capacity, features and walking time all pass"]
```

```
stateDiagram-v2
[*] --> CONFIRMED : faculty reserves, room free and capacity fits
[*] --> PENDING_APPROVAL : seminar hall, library, or event above 100
PENDING_APPROVAL --> CONFIRMED : administrator approves
PENDING_APPROVAL --> REJECTED : administrator rejects
PENDING_APPROVAL --> CANCELLED : requester withdraws
CONFIRMED --> CONFIRMED : reschedule, new interval revalidated
CONFIRMED --> COMPLETED : end time passes
CONFIRMED --> CANCELLED : cancel with mandatory reason
CONFIRMED --> DISPLACED : administrator override for an examination
DISPLACED --> CANCELLED : new room accepted
note right of CANCELLED
No transition deletes a row. The interval becomes reusable because the
exclusion constraint only covers CONFIRMED and PENDING.
end note
```

Appendix C. Worked derivation of the clash probability

The estimate in section 8.3 is worth deriving once, so that it can be checked or replaced.

Let M be the number of distinct room-terms available, here thirty rooms across eight periods, so M equals two hundred and forty. Let N be the number of independent booking attempts in a day, each choosing a room-term uniformly at random, which is the simplifying assumption. The number of unordered pairs of attempts is N(N−1)/2, and each pair collides with probability 1/M. By linearity of

expectation the expected number of colliding pairs is

$$E = N\,(N - 1)\,/\,2M$$

For thirty attempts and two hundred and forty room-terms, E equals one point eight one. Treating the collisions as approximately Poisson with that mean gives the probability of at least one collision as one minus e to the minus E, which is zero point eight four, or eighty-four per cent. Over eighteen weeks of six days, the expected number of colliding pairs is one point eight one multiplied by one hundred and eight, or about one hundred and ninety-six.

The assumptions are generous in one direction and stingy in another. Uniform choice makes collisions less likely than reality, because demand concentrates on popular rooms and periods; independence likewise understates the problem, because the same person books the same slot for a series. Against that, the model ignores conflicts discovered and resolved verbally before anyone writes them down. We regard the figure as a floor, and we present it as an order-of-magnitude argument rather than as a prediction. The point it supports does not depend on the exact number: uncoordinated booking produces collisions at a rate that rises steeply with volume, and a constraint in the database reduces that rate to zero rather than to a smaller number.

Appendix D. Glossary

Term	Definition
Half-open interval	A range that includes its start instant and excludes its end, written [start, end). The convention that makes back-to-back bookings legal.
Exclusion constraint	A rule enforced by PostgreSQL that refuses any row whose keys conflict under a stated operator, used here to make overlapping live reservations unrepresentable.
Displacement	An administrative override that moves an existing reservation to free a room, recorded with a reason and accompanied by a notice.
Outbox	A table of pending notifications written inside the reservation transaction and delivered afterwards by a scheduled job.
Spaced capacity	The number of candidates a room holds with examination spacing, here taken as half its nominal capacity.
Idempotency key	A client-supplied value that lets a repeated request return the original result instead of creating a second record.
Room-term	One room in one period; the unit of supply in the arithmetic of section 9.
Section	A teaching group of about seventy-five students, for example CSE-3A.
Unconfirmed	A flag on a room whose attributes were inferred rather than stated in the supplied material.

TABLE 29 *Terms as used in this document, in the order they first matter.*

End of document.